

Test Bench & Simulator

Module Convenor: Dr.T. Nandha Kumar

Email: nandhakumaar.t@nottingham.edu.my

Department of Electrical & Electronic Engineering
The University of Nottingham Malaysia Campus

Objectives

After completing this module, you will be able to:

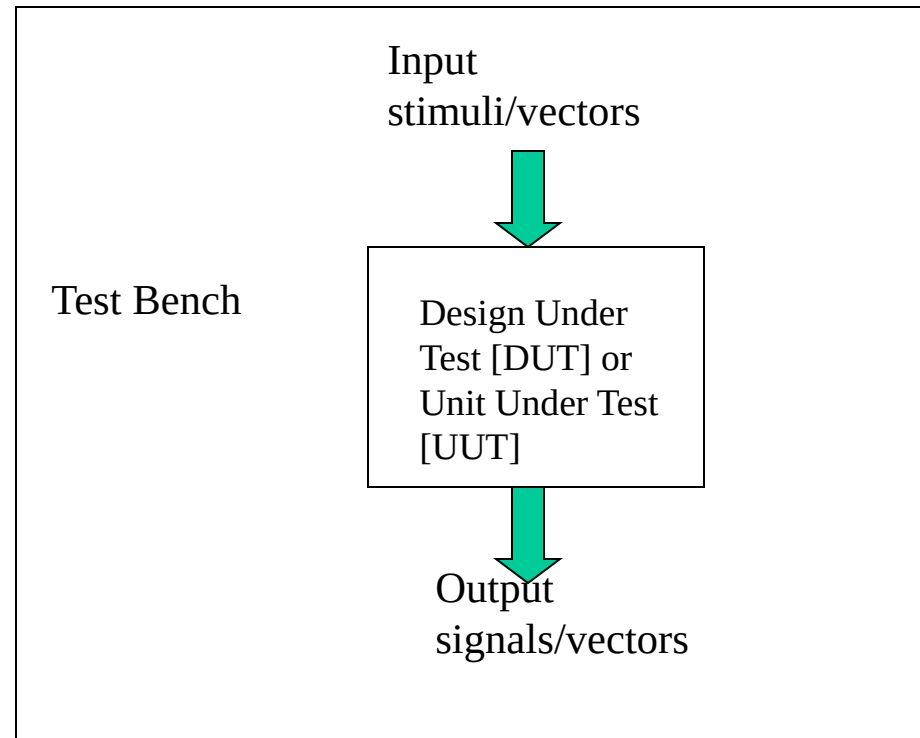
- Understand the different VHDL constructs used in developing testbench code.
- To be able to write a medium complexity testbench for a digital design.
- Understand the various methods used in pre silicon validation.
- Understand the Modelsim and Xilinx XST tool flow in generating testbench and simulating the design

Overview

- A model used to exercise and verify the correct functionality of a digital design that is written in a HDL code
- The main purposes are
 - To generate the input stimulus for simulation
 - To apply the stimuli to the design
 - To verify the output responses of the design under test with the expected values.
- In hardware terms:
 - “stimulus generation” could be a signal generator
 - “output verification” could be a logic analyser

Overview (cont...)

- A testbench generates the input stimuli for the DUT and verifies the output signals from the DUT
- For the test bench the input stimuli and output signals are internal signals
- For the DUT they are external signals
- The VHDL testbench doesn't have to be synthesizable
- It is recommended that a validation engineer develops the test bench rather than a design engineer.



Overview (cont...)

- The test bench can be either:
 - simple, providing a sequence of inputs to the circuit over time
 - sophisticated like reading/writing test data from an input/output text file
- A comprehensive test bench can in fact take longer to develop than the design under test itself!!
- A testbench comprise of
 - Entity
 - Generally empty
 - Architecture
 - Wires to connect the UUT and the testbench or the driver model
 - Instantiates the drivers
 - Configuration
 - Binding information of UUT version and testbench version

Test Bench : case study 1

- VHDL code for an OR gate is

```
31  entity orgate is
32      port(a : in std_logic_vector(1 downto 0);
33           y : out std_logic);
34  end orgate;
35
36  architecture orgate_arch of orgate is
37
38  begin
39
40      y <= a(1) or a(0);
41
42  end orgate_arch;
43
```

Test Bench : case study 1 (cont...)

orgate Testbench Template:

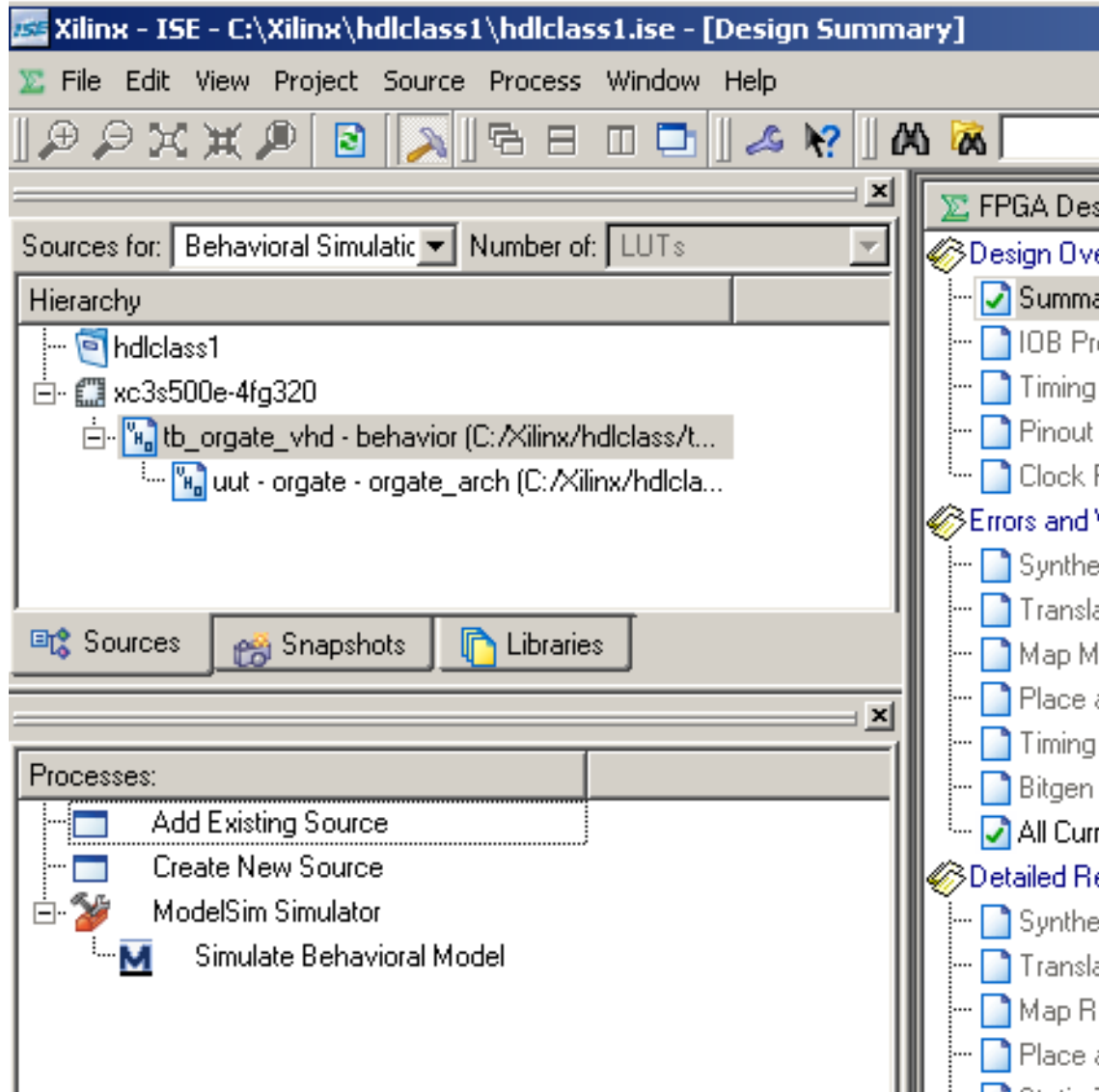
```
28 -----
29 LIBRARY ieee;
30 USE ieee.std_logic_1164.ALL;
31 USE ieee.std_logic_unsigned.all;
32 USE ieee.numeric_std.ALL;
33
34 ENTITY tb_orgate_vhd IS
35 END tb_orgate_vhd;
36
37 ARCHITECTURE behavior OF tb_orgate_vhd IS
38
39     -- Component Declaration for the Unit Under Test (UUT)
40     COMPONENT orgate
41     PORT(
42         a : IN std_logic_vector(1 downto 0);
43         y : OUT std_logic
44     );
45     END COMPONENT;
46
47     --Inputs
48     SIGNAL tb_a : std_logic_vector(1 downto 0) := (others=>'0');
49
50     --Outputs
51     SIGNAL tb_y : std_logic;
52
53 BEGIN
54
55     -- Instantiate the Unit Under Test (UUT)
56     uut: orgate PORT MAP(
57         a => tb_a,
58         y => tb_y
59     );
60
61     tb : PROCESS
62     BEGIN
63
64         -- Wait 100 ns for global reset to finish
65         wait for 100 ns;
66
67         -- Place stimulus here
68
69         wait; -- will wait forever
70     END PROCESS;
71
72 END;
```

Test Bench : case study 1 (cont...)

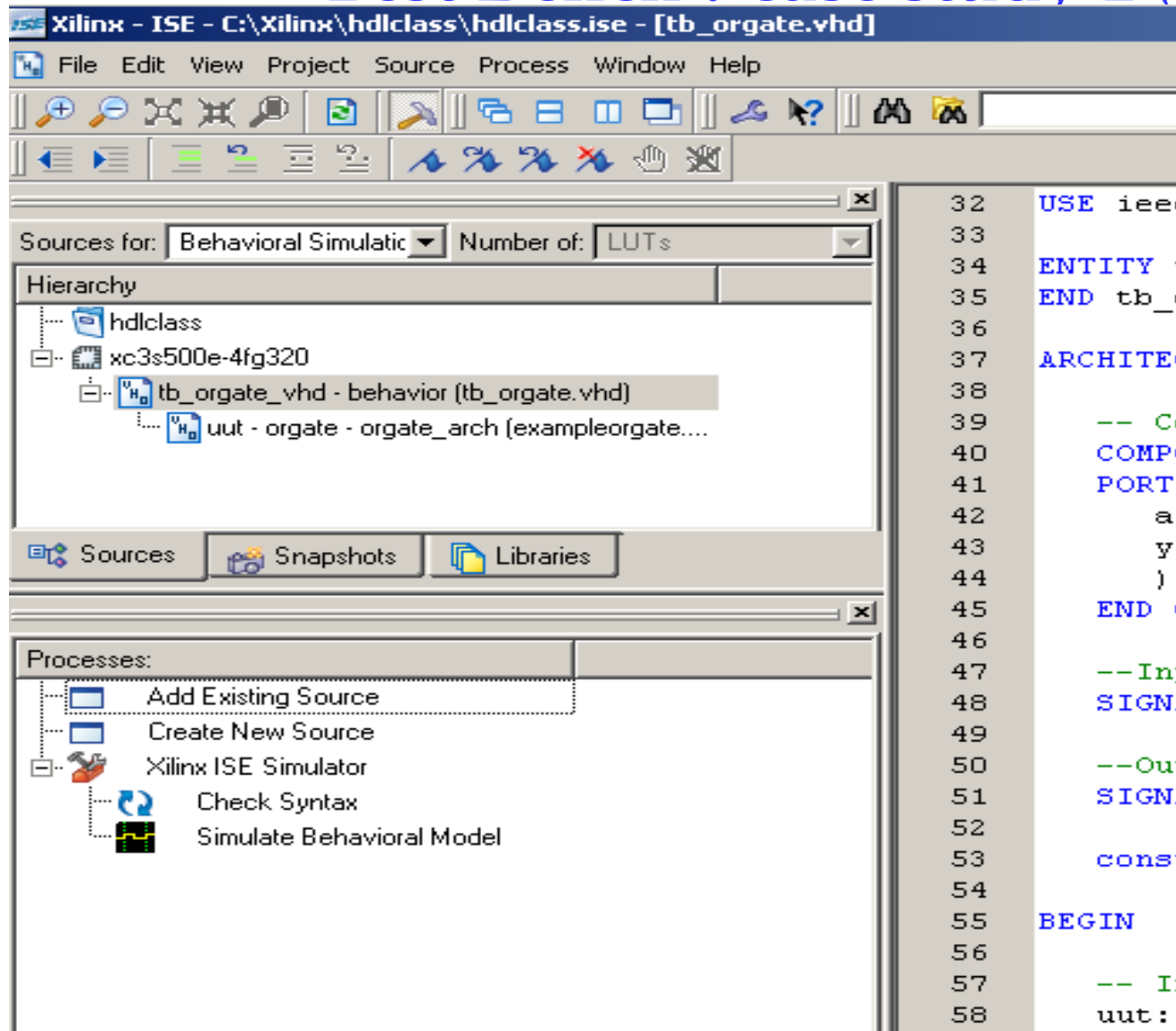
orgate Testbench:

```
34 ENTITY tb_orgate_vhd IS
35 END tb_orgate_vhd;
36
37 ARCHITECTURE behavior OF tb_orgate_vhd IS
38
39     -- Component Declaration for the Unit Under Test (UUT)
40     COMPONENT orgate
41     PORT(
42         a : IN std_logic_vector(1 downto 0);
43         y : OUT std_logic
44     );
45 END COMPONENT;
46
47 --Inputs
48 SIGNAL tb_a : std_logic_vector(1 downto 0) := (others=>'0');
49
50 --Outputs
51 SIGNAL tb_y : std_logic;
52
53 constant period : time := 50ns;
54
55 BEGIN
56
57     -- Instantiate the Unit Under Test (UUT)
58     uut: orgate PORT MAP(
59         a => tb_a,
60         y => tb_y
61     );
62
63     tb : PROCESS
64     BEGIN
65
66         -- Wait 100 ns for global reset to finish
67         wait for 100 ns;
68
69         -- Place stimulus here
70         tb_a<="00"; wait for period; tb_a<="01"; wait for period;
71         tb_a<="10"; wait for period; tb_a<="11"; wait for period;
72
73         wait; -- will wait forever
74     END PROCESS;
75
76 END;
```

Test Bench : case study 1 (cont...)



Test Bench : case study 1 (cont...)



The screenshot shows the Xilinx ISE environment. The top menu bar includes File, Edit, View, Project, Source, Process, Window, and Help. Below the menu is a toolbar with various icons for file operations and simulation. The main workspace is divided into three panes:

- Sources for:** Behavioral Simulation. Number of: LUTs.
- Hierarchy:** Shows the project structure:
 - hdlclass
 - xc3s500e-4fg320
 - tb_orgate_vhd - behavior (tb_orgate.vhd) (selected)
 - uut - orgate - orgate_arch (exampleorgate....)
- Processes:**
 - Add Existing Source
 - Create New Source
 - Xilinx ISE Simulator
 - Check Syntax
 - Simulate Behavioral Model

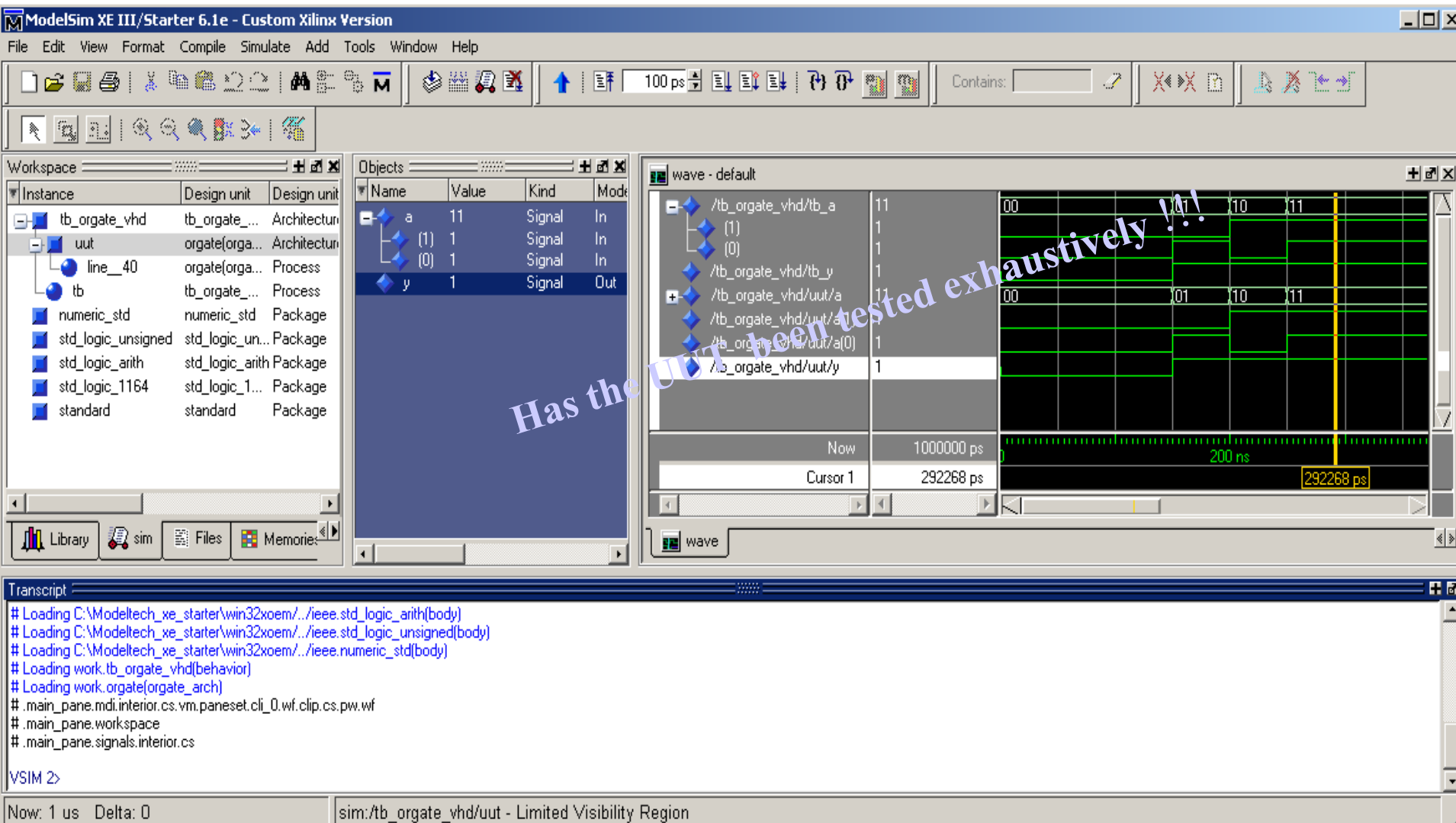
The right pane displays the VHDL code for the test bench:

```

32  USE ieee
33
34  ENTITY tb_
35  END tb_
36
37  ARCHITE
38
39      -- C
40  COMP
41  PORT
42      a
43      y
44  )
45  END
46
47  -- In
48  SIGN
49
50  -- Ou
51  SIGN
52
53  cons
54
55  BEGIN
56
57  -- I:
58  uut :
```

Test Bench : case study 1 (cont...)

Snapshot of the simulator result



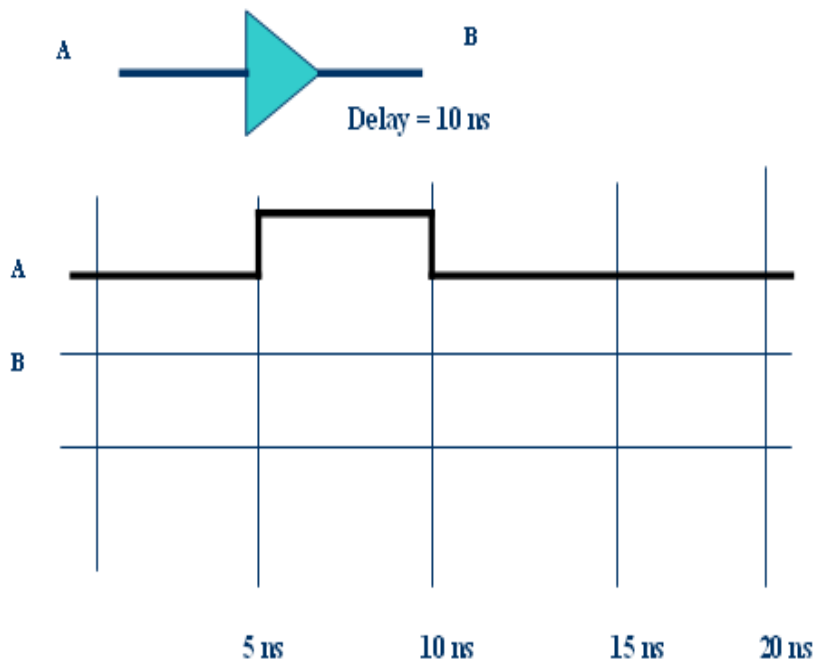


VHDL for Test Bench Generation

- Let's focus on the general VHDL constructs used in Testbench design to model
 - **Time and delay**
 - **Warning**
 - **Waveforms**

Time and Delay

- Two types
 - Inertial Delay or Component Delay
 - Transport Delay or Wire Delay
- Inertial or Component Delay



```
entity buffer_delay is
    Port ( a : in std_logic;
           b : out std_logic);
```

```
end buffer_delay;
```

```
architecture Behavioral of buffer_delay is
```

```
begin
```

```
    b <= a after 10ns;
```

```
end Behavioral;
```

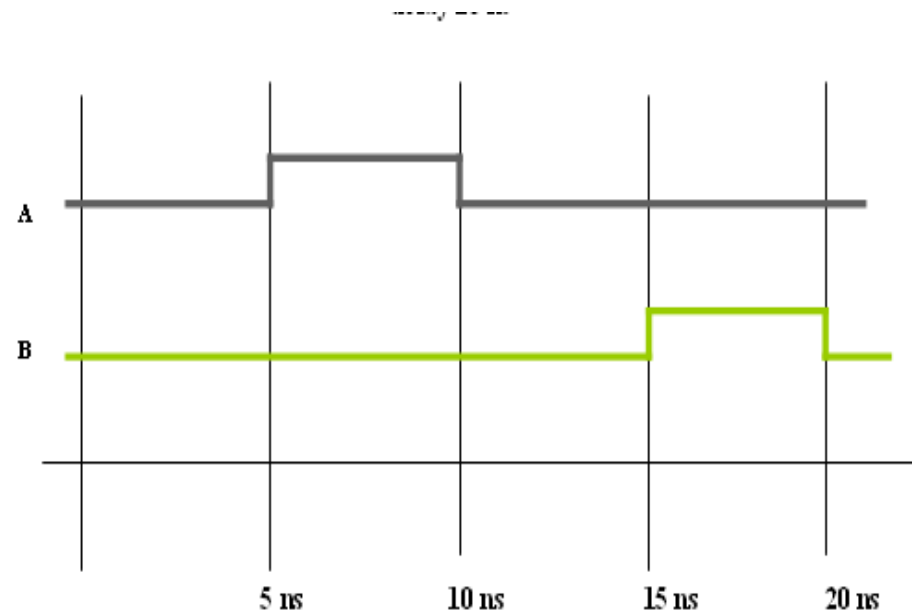
Time and Delay

- Inertial or Component Delay
 - Delay due to the output signal inertia of a device
 - It is the delay through the device
 - Determined by the technology
 - If the pulse width of the input signal is less than the inertial delay then no change occurs on the output signal.

Time and Delay

- Transport Delay
 - To model the interconnect delay or wire delay
 - This delay is the time taken by a signal to propagate from one point to another through a wire.
 - Any pulse is transmitted, no matter on its width.
 - It uses a key word “transport” in the signal assignment statement

$b \leq a$ transport after 10 ns;



Time and Delay

- Both transport & after are non synthesizable code.
- Actual timing will be incorporated in the timing model or gate level model.
- Reject
 - Command used to specify the length of the pulse to propagate through the component.
 - `b <= reject 6ns inertial a after 8ns;`
 - Allow all the pulses greater than 6ns to propagate through the component.
 - reject uses inertial key word
- Delta Delay
 - A small delay happening at the same clk
 - delay between a clk tick to another

Time and Delay

- “Null” statement
 - do nothing
- “Wait” statement
 - Used to suspend the sequential execution of a process
 - Four ways to describe wait statements
 - process (a, b)
 - wait for --- time
 - wait on --- signal change
 - wait until --- expression is true

Time and Delay

- Wait on

- Wait ON set, reset ;

The process will wait till there is any changes in the signals set or reset

- Wait until expression

eg. 1

```
process
begin
wait until clock = '1';
      q <= d ;
end process;
```

eg.2

```
wait until ((x * 8) >= 80);
```

Time and Delay

- Wait for time_expression
 - Eg.
 - wait for 10 ms;
 - wait for ((a *b) – c);

Multiple wait Expression

All the wait statements can be combined to form an multiple wait statement

eg.

wait on set, en until ((set = '1') or (en = '1')) for 10 ns ;

Time and Delay

Delay constant declaration:

- **Constant** identifier: time:= value;

Note:

- Time is a VHDL built-in data type
- Value can be 1 ms, 7 ns, 100 ps, etc

- How to prevent indeterminate waiting?
 - Eg . Wait until a = ‘1’;

Here execution stops and waits till a value changes to ‘1’

- It is essential to alert the testing engineer with a message
- Assert can be used as a wait time out statement

Warning

- Assert

- Used to report information
- Checks the Boolean expression
 - If true, does nothing
 - If false, reports at the output
- Four different levels of severity are
 - Note
 - Warning
 - Error
 - Failure (stops the simulation)
- Non synthesizable code

```
wait until (a = '1') for 10 us;  
assert ( a = '1' )  
report “ a did not change to  
the value to '1' & time out”  
severity error;
```

- **Syntax**

Assert < condition>

Report <message>

Severity < error_level>

Assert Examples

`assert (not (set = '1' and reset = '1'))`

Report “Set and Reset both asserted”

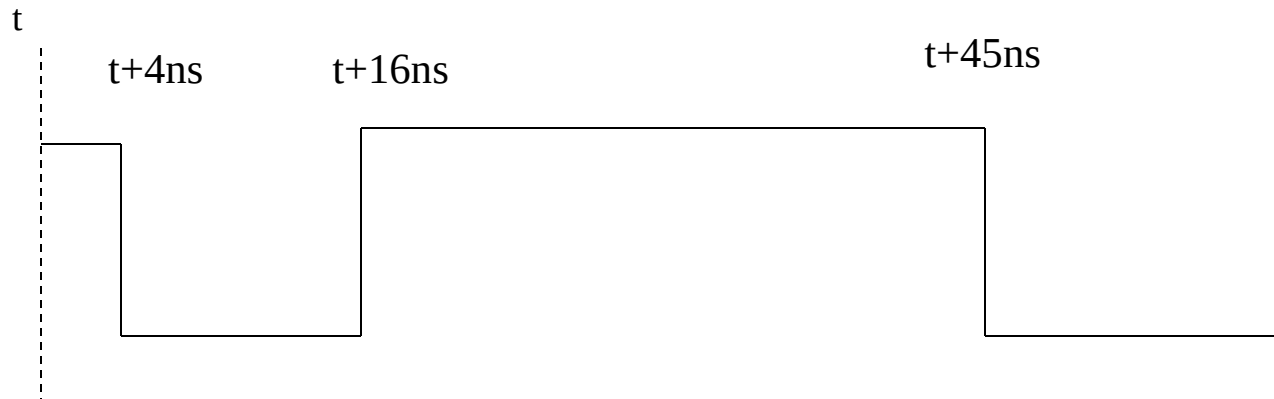
Severity Error;

`assert (set = '1' or reset = '1')`

SETUP and HOLD TIME

Waveform

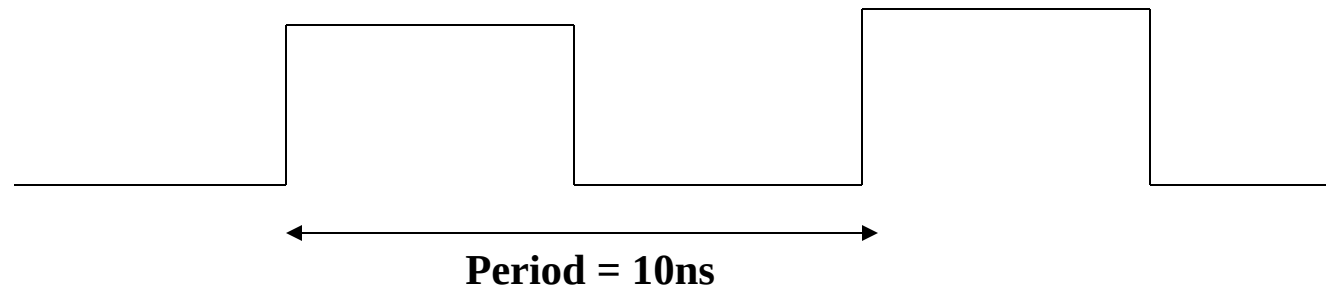
- Arbitrary wave form
 $y \leq '1'$, '0' after 4ns, '1' after 16 ns, '0' after 45ns;



Waveform

- Periodic Waveform

```
architecture periodic of wave is  
    signal clk :bit := '0';  
  
begin  
    clk <= NOT clk after 5 ns;  
end periodic;
```



Waveform

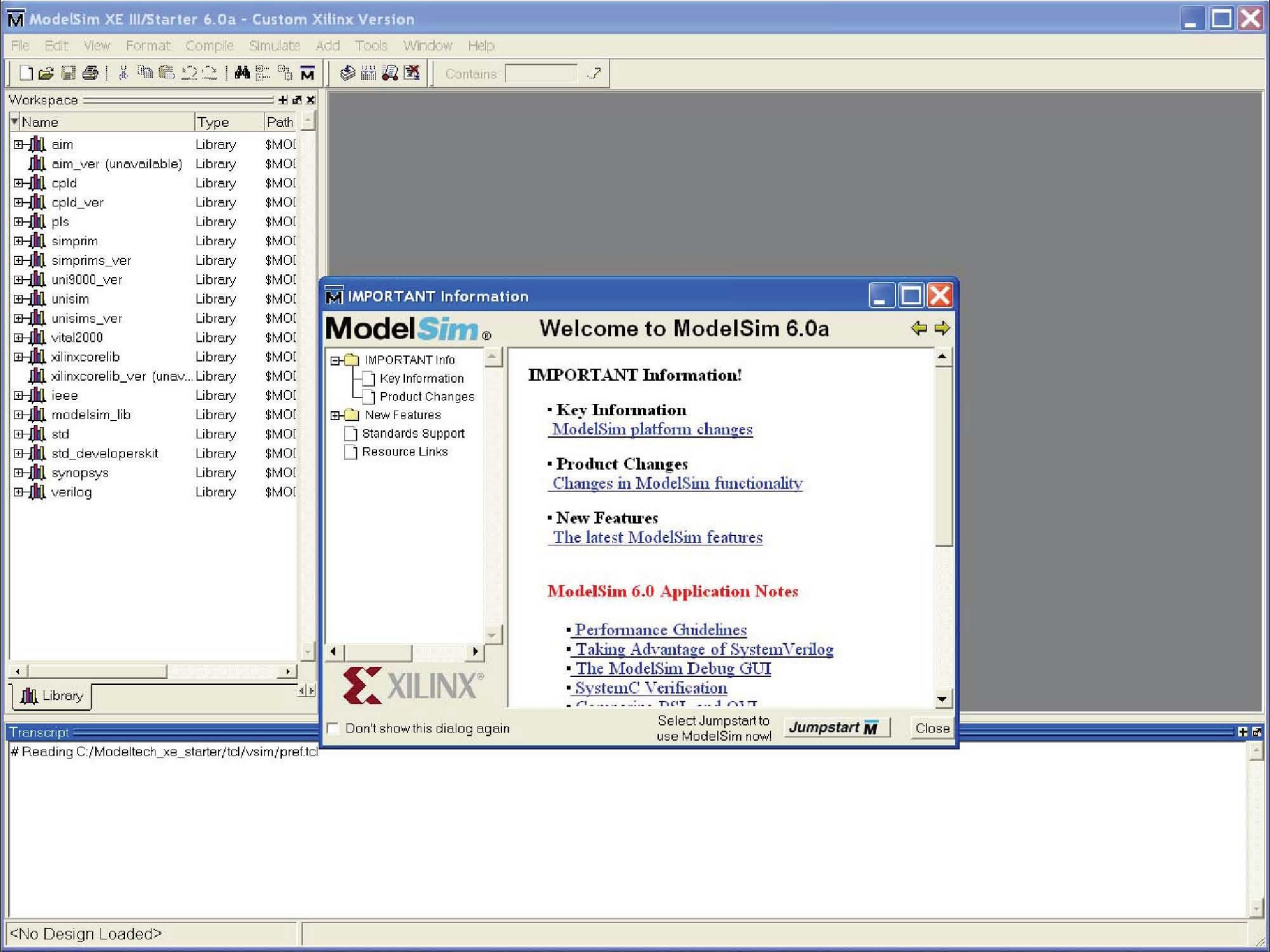
- Periodic Waveform

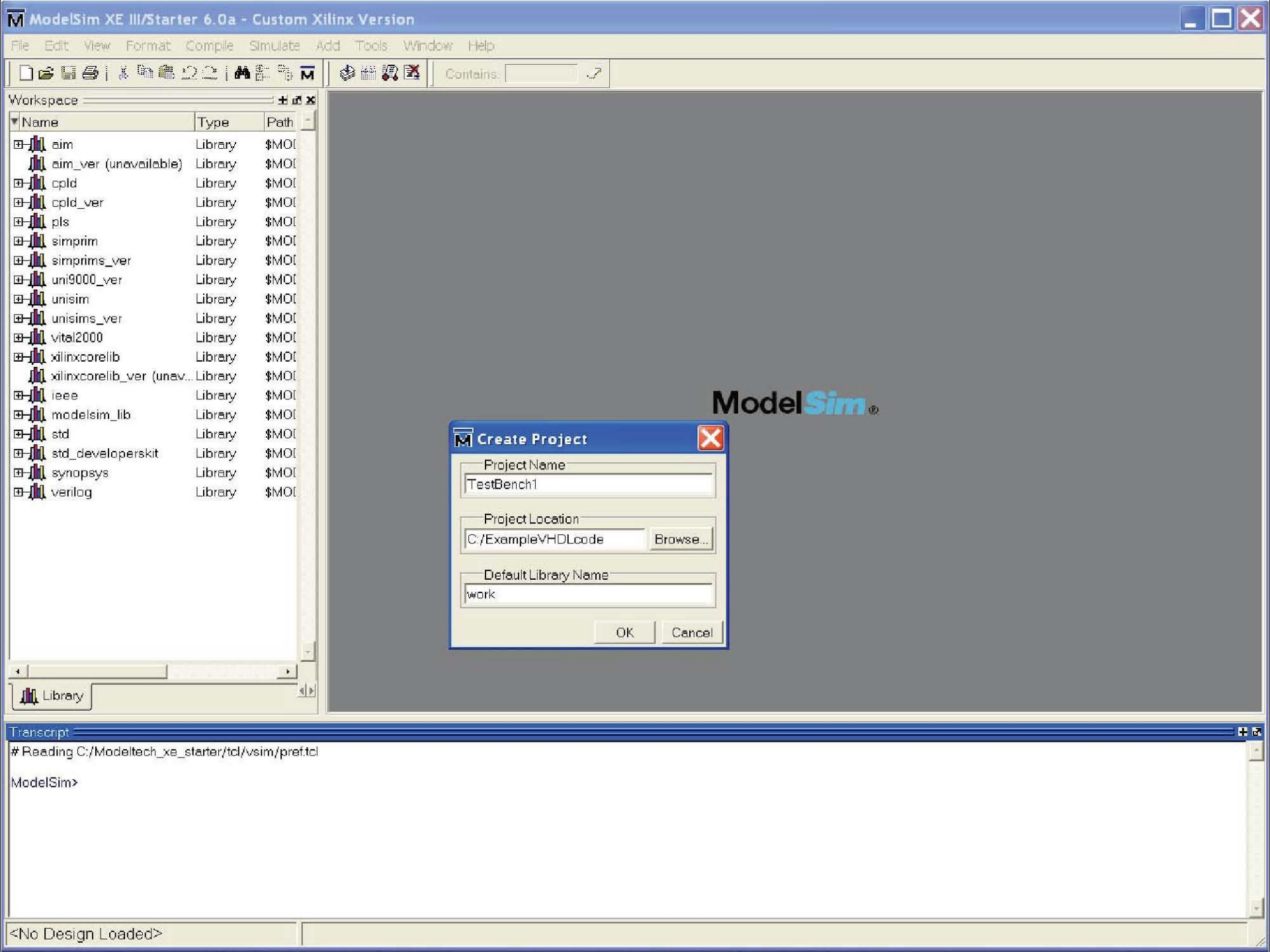
```
architecture periodic of wave is  
  
begin  
    process  
    begin  
        clk <= '0';  
        wait for 5ns;  
        clk <= '1';  
        wait for 5ns;  
    end process;  
  
end periodic;
```

Waveform

- Write a VHDL code to generate two periodic clock signals each with 20ns period, but out of phase by 5ns.

Testbench with Modelsim





Name	Type	Path
aim	Library	\$MOI
aim_ver (unavailable)	Library	\$MOI
cpld	Library	\$MOI
cpld_ver	Library	\$MOI
pls	Library	\$MOI
simprim	Library	\$MOI
simprims_ver	Library	\$MOI
uni9000_ver	Library	\$MOI
unisim	Library	\$MOI
unisims_ver	Library	\$MOI
vital2000	Library	\$MOI
xilinxcorelib	Library	\$MOI
xilinxcorelib_ver (unav...	Library	\$MOI
ieee	Library	\$MOI
modelsim_lib	Library	\$MOI
std	Library	\$MOI
std_developerskit	Library	\$MOI
synopsys	Library	\$MOI
verilog	Library	\$MOI

Library

ModelSim®

Create Project

Project Name

TestBench1

Project Location

C:/ExampleVHDLcode

Browse...

Default Library Name

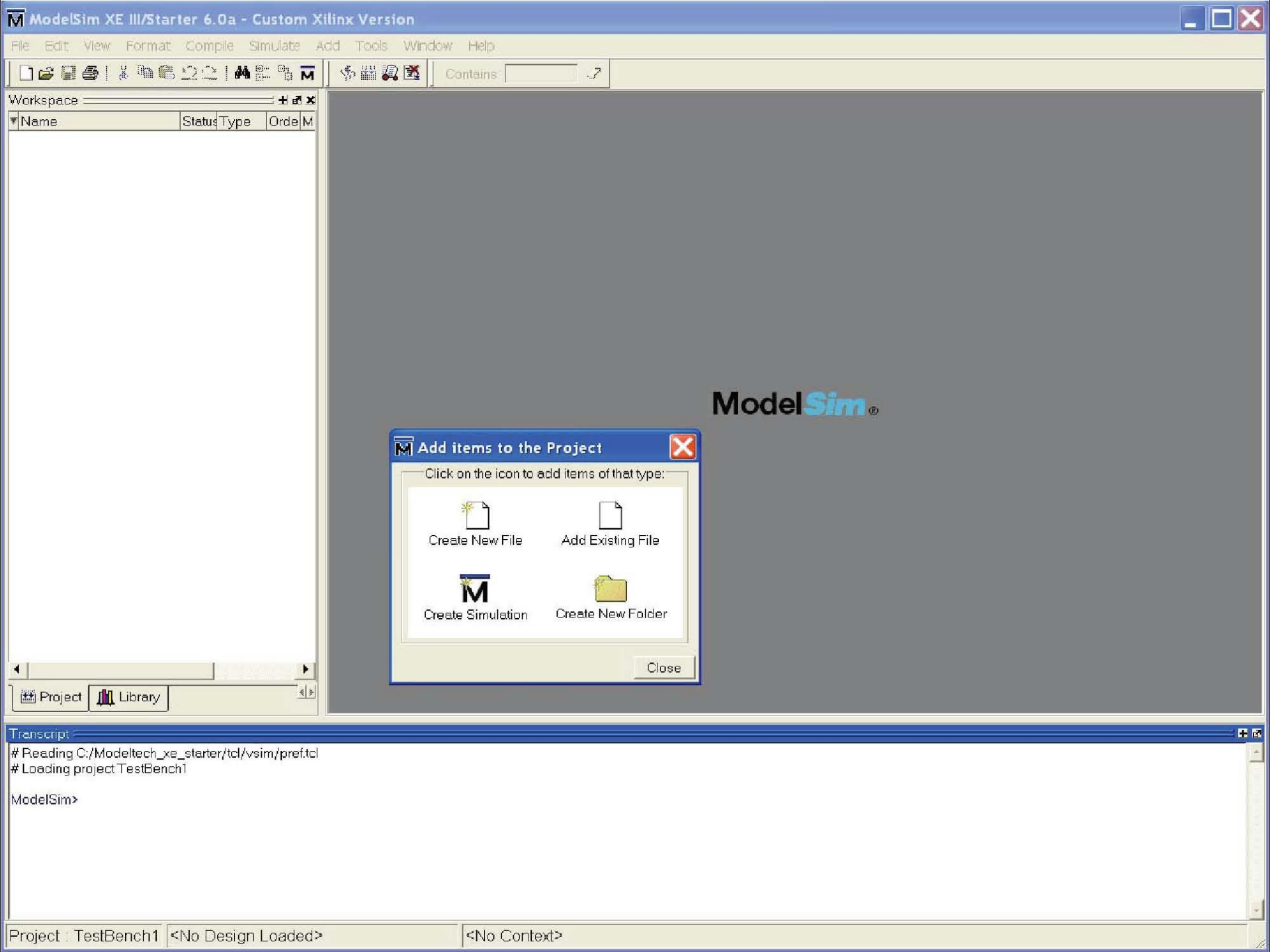
work

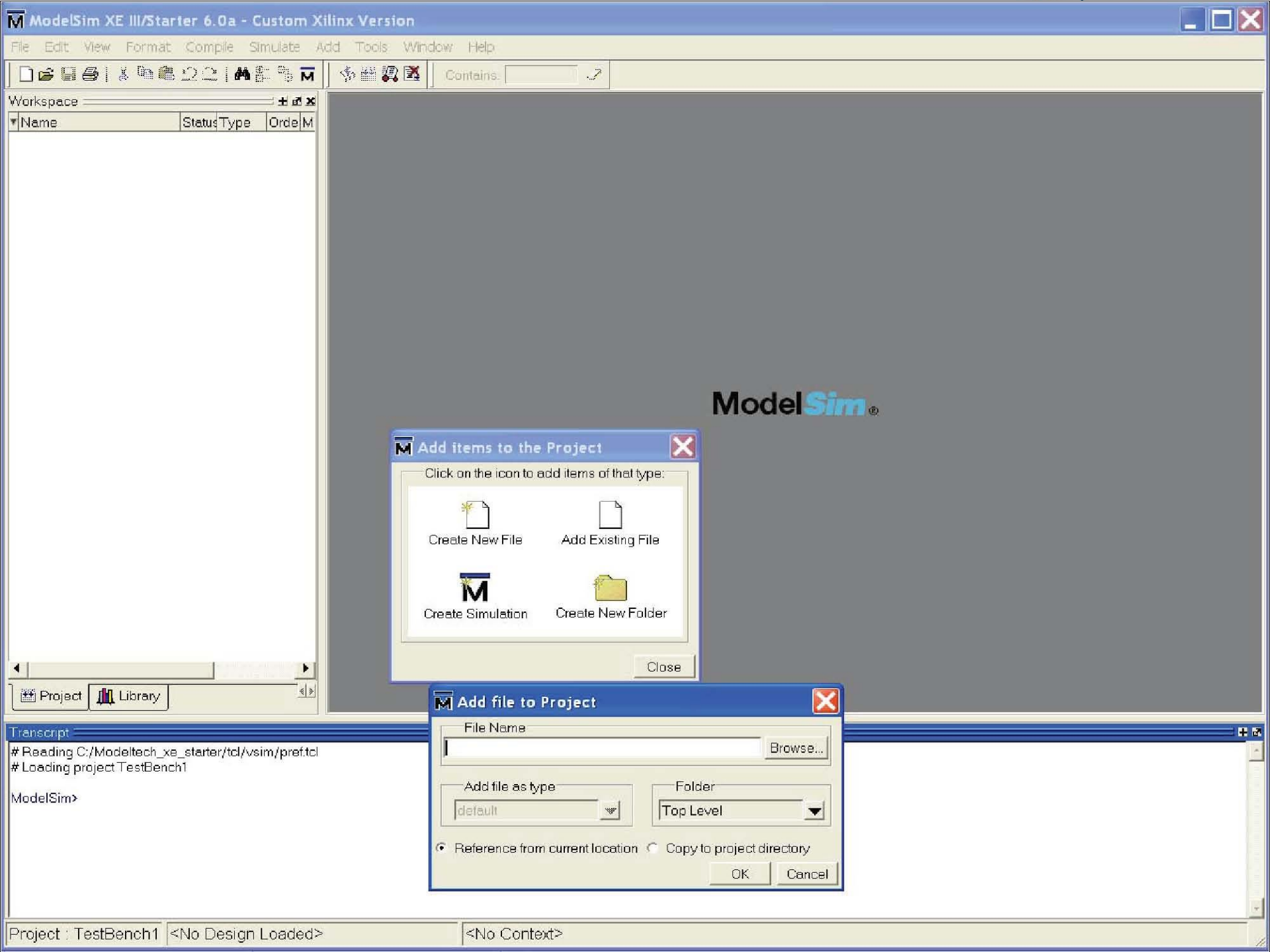
OK

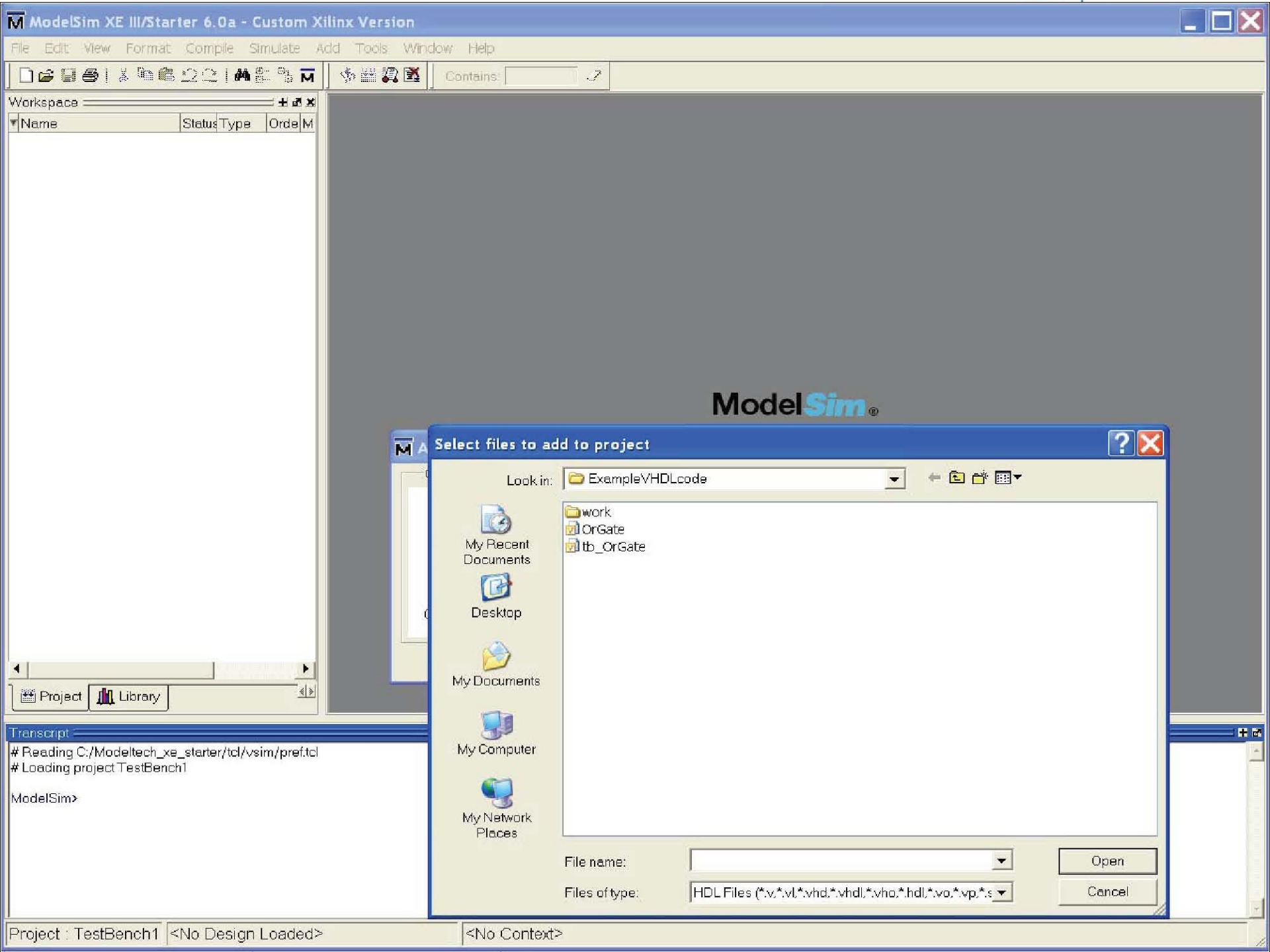
Cancel

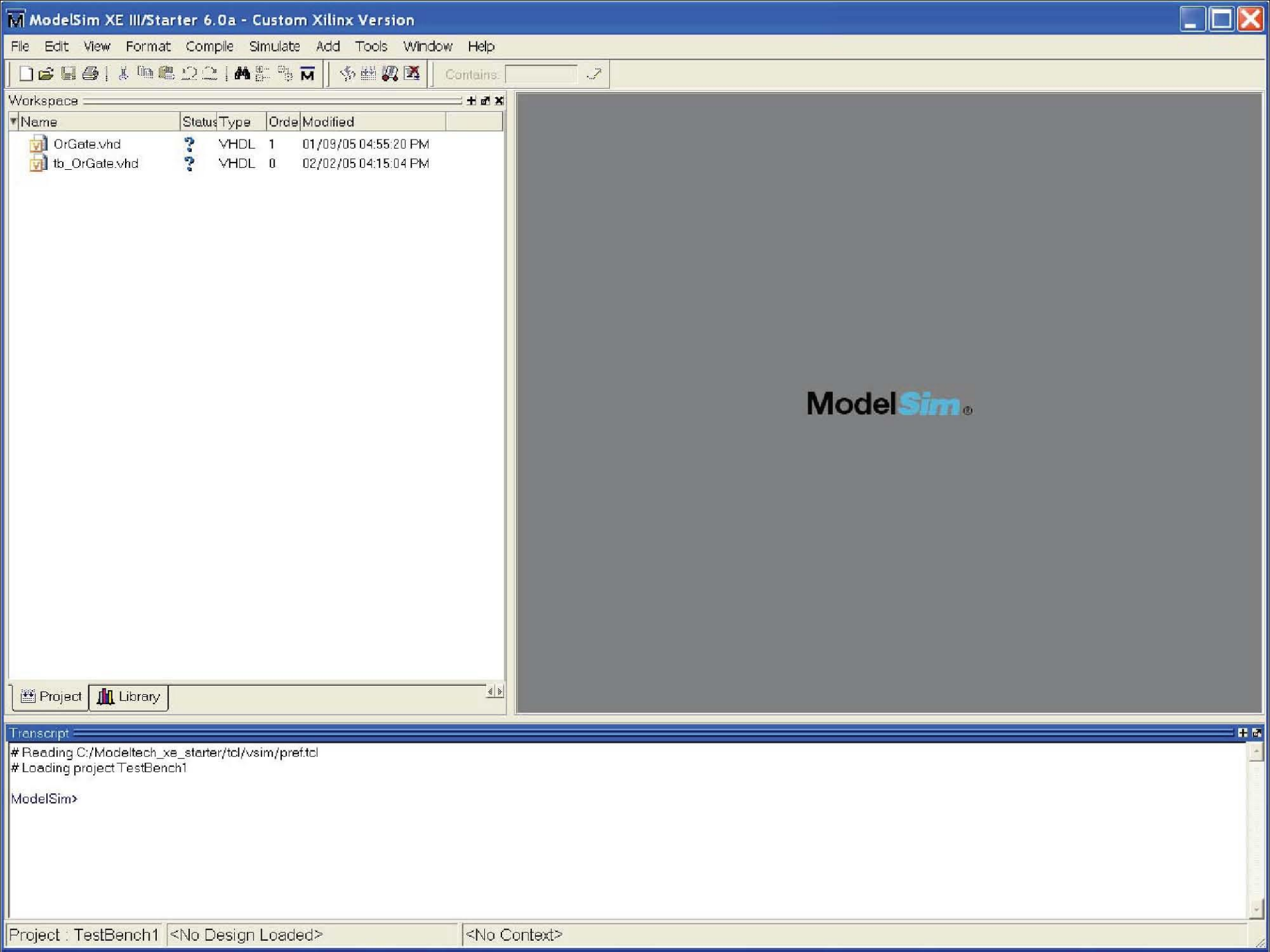
Reading C:/Modeltech_xe_starter/tcl/vsim/pref.tcl

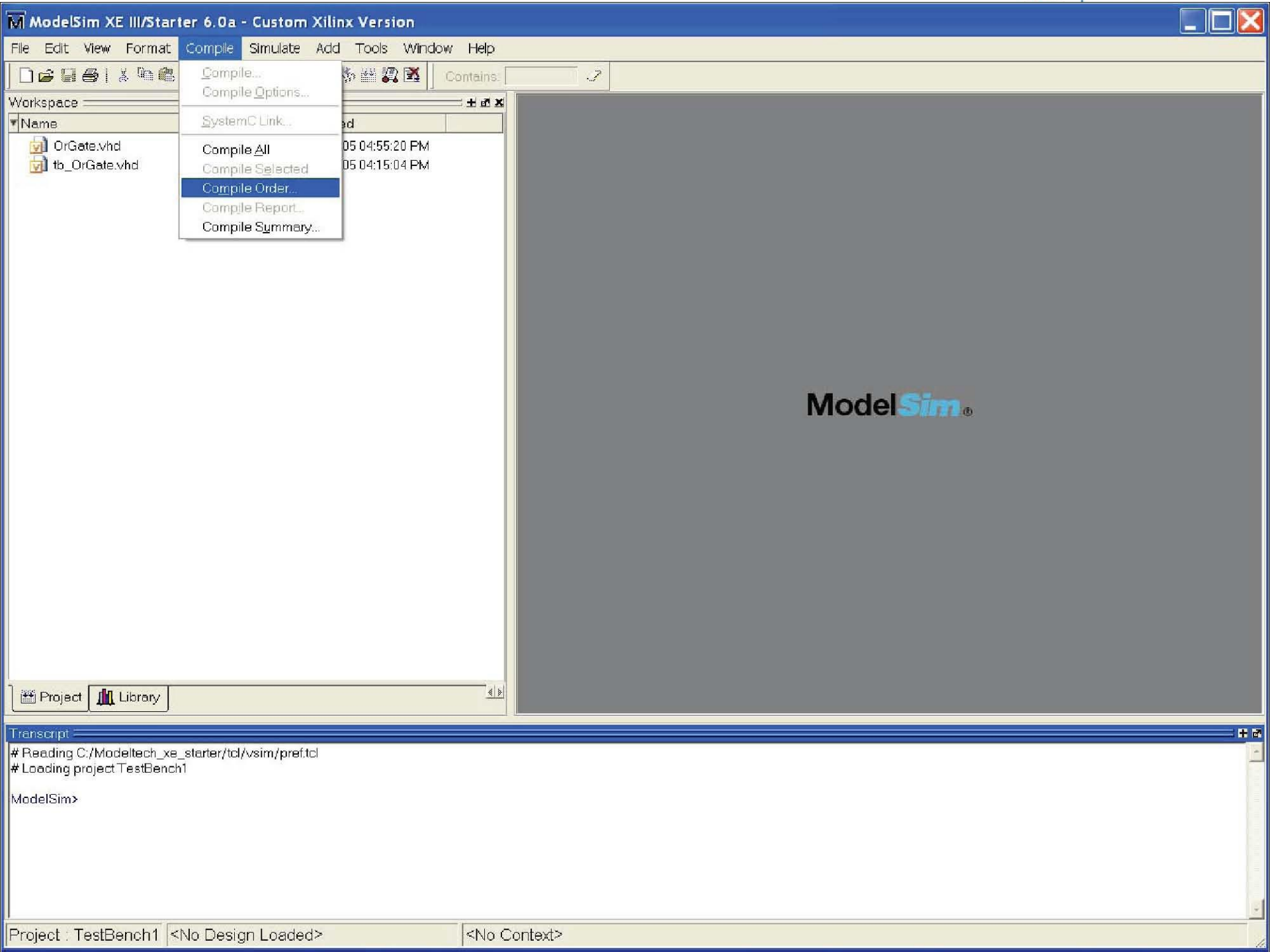
ModelSim>

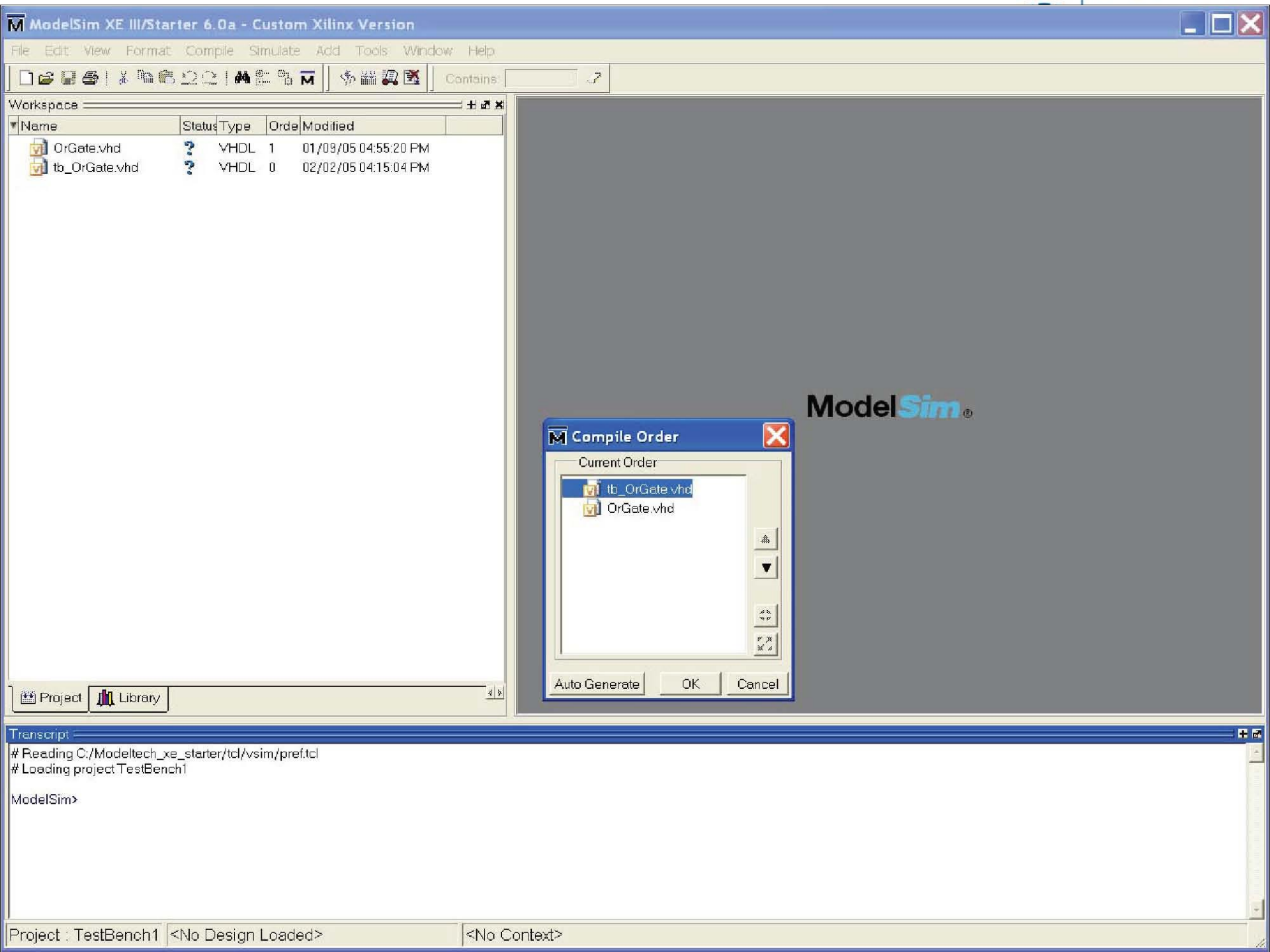


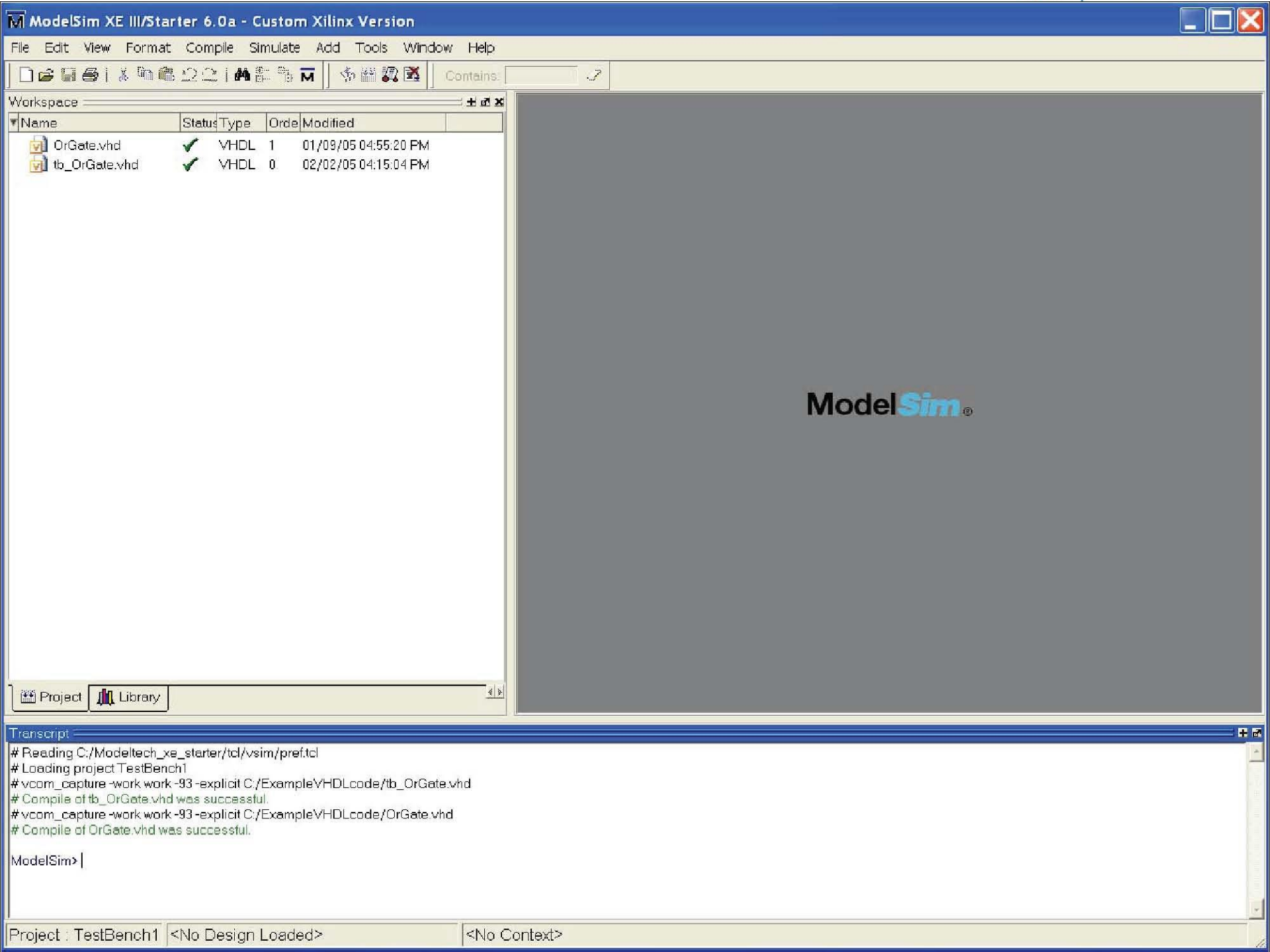


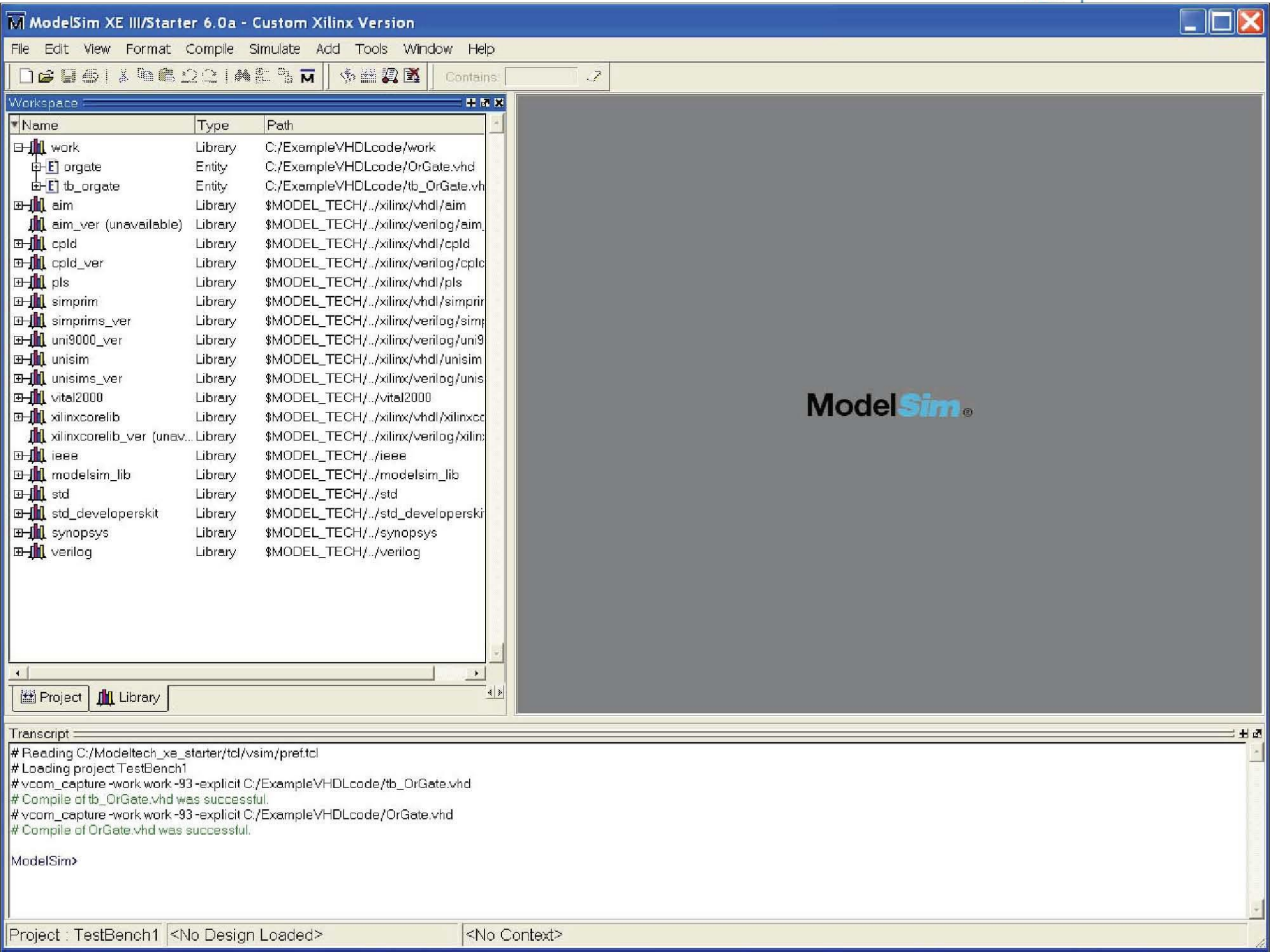
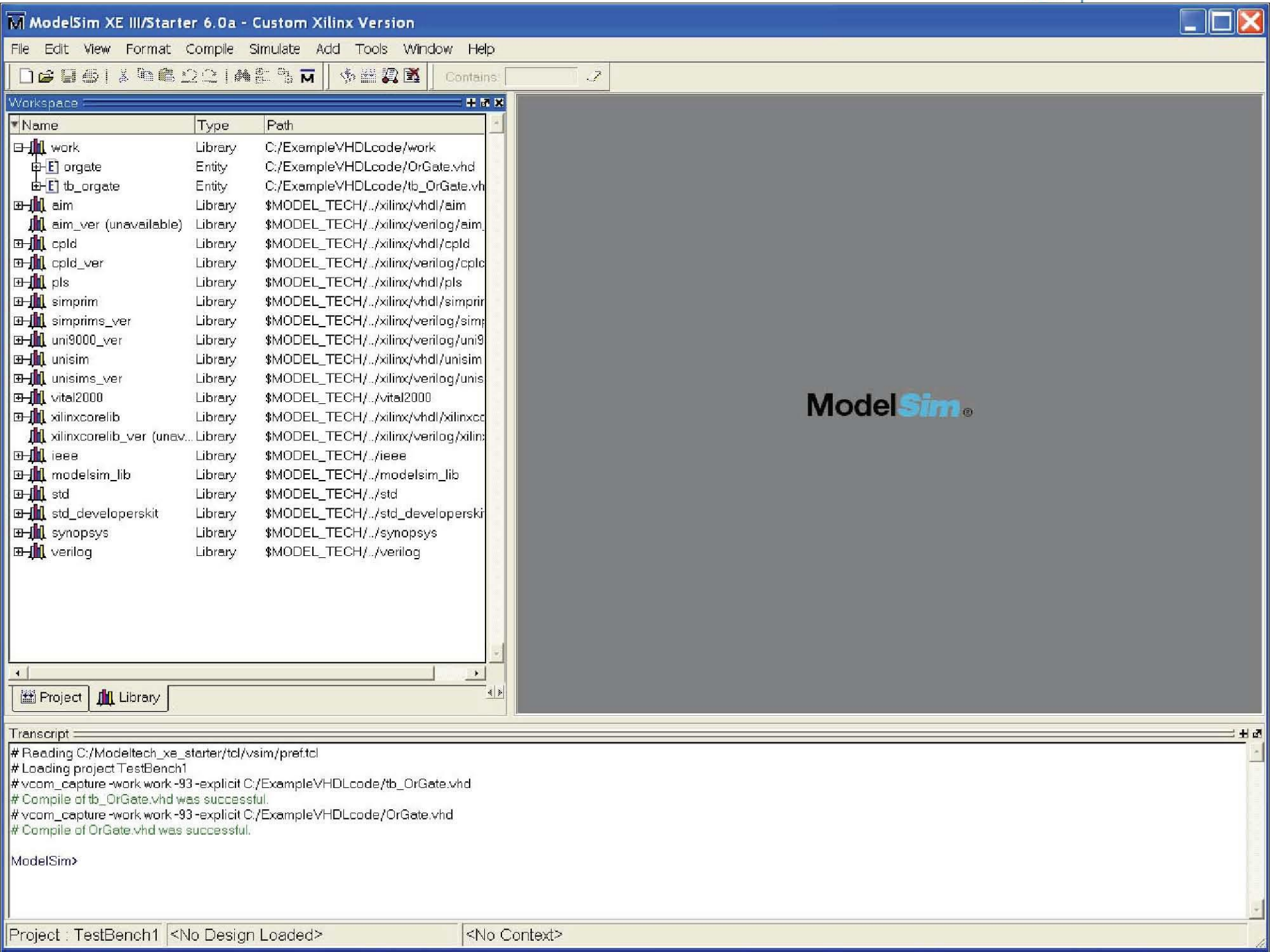


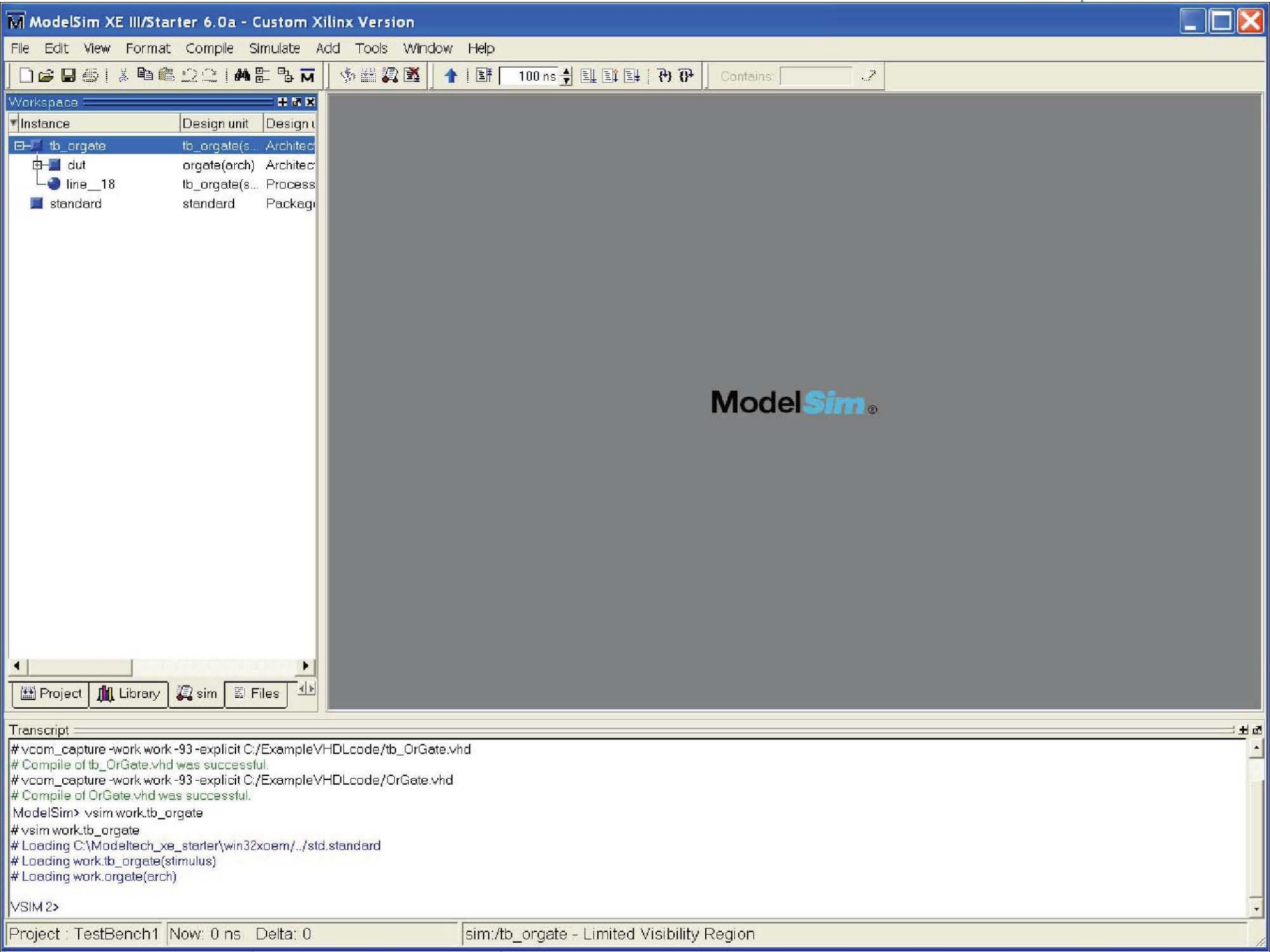


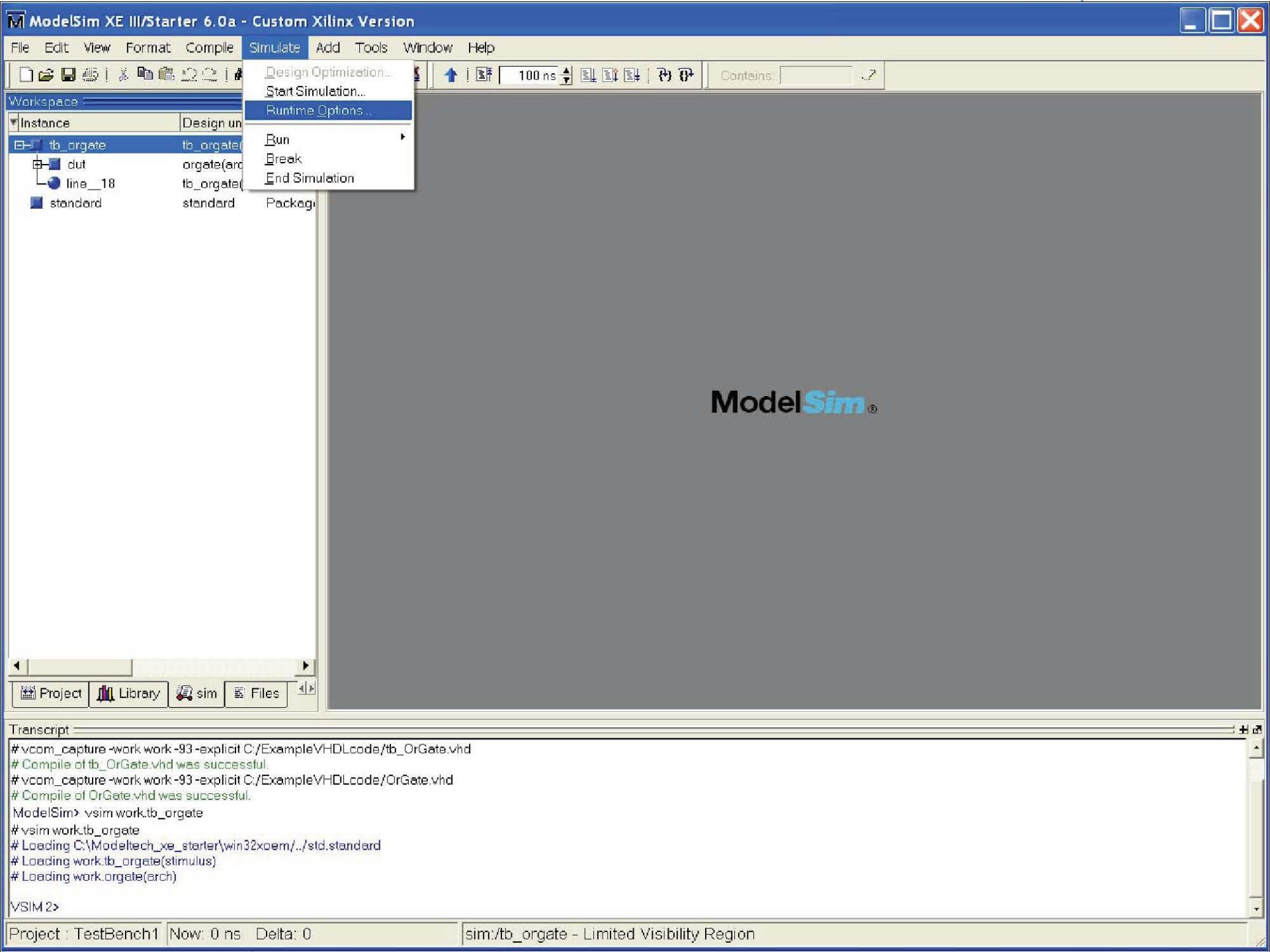


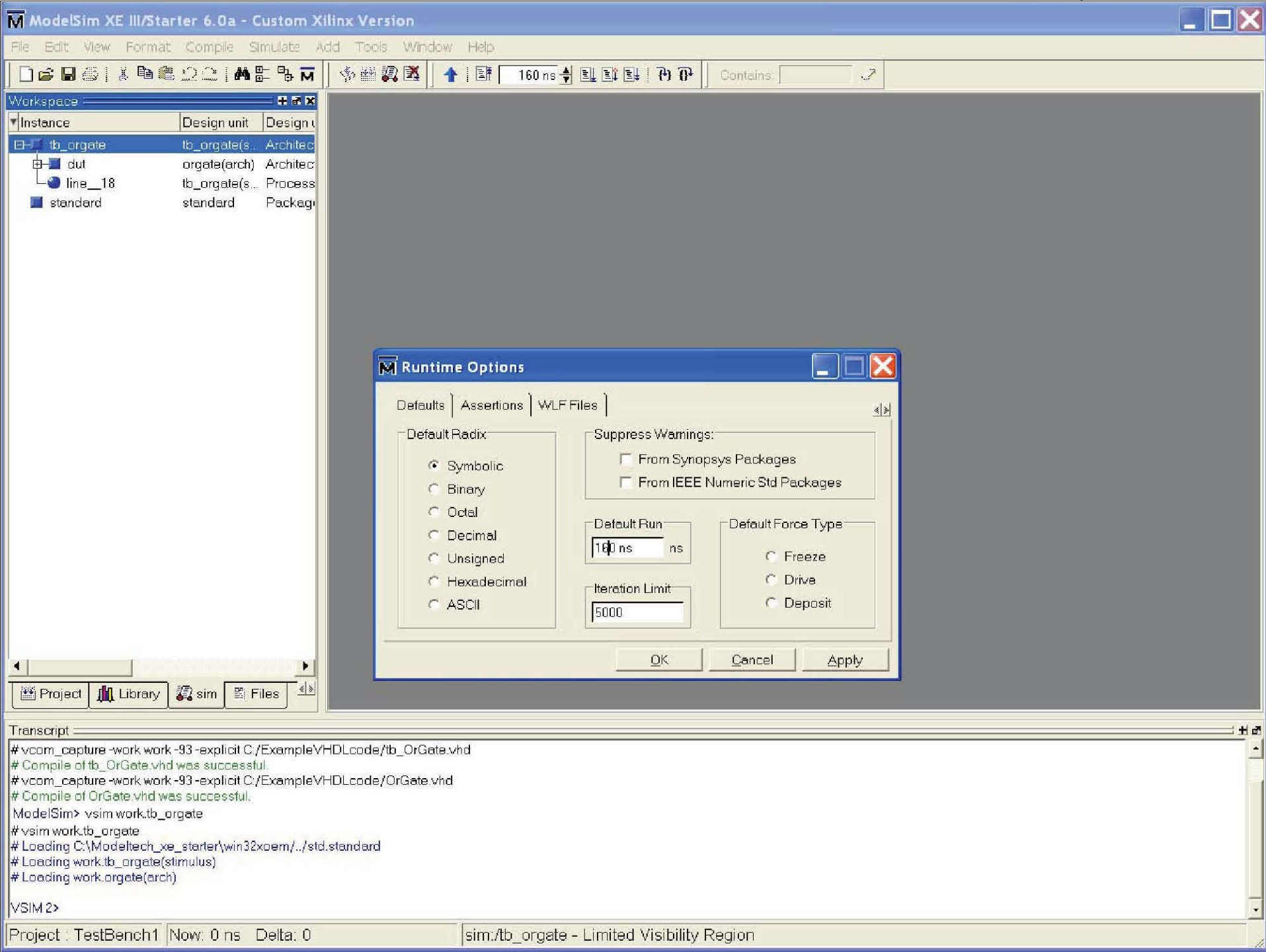


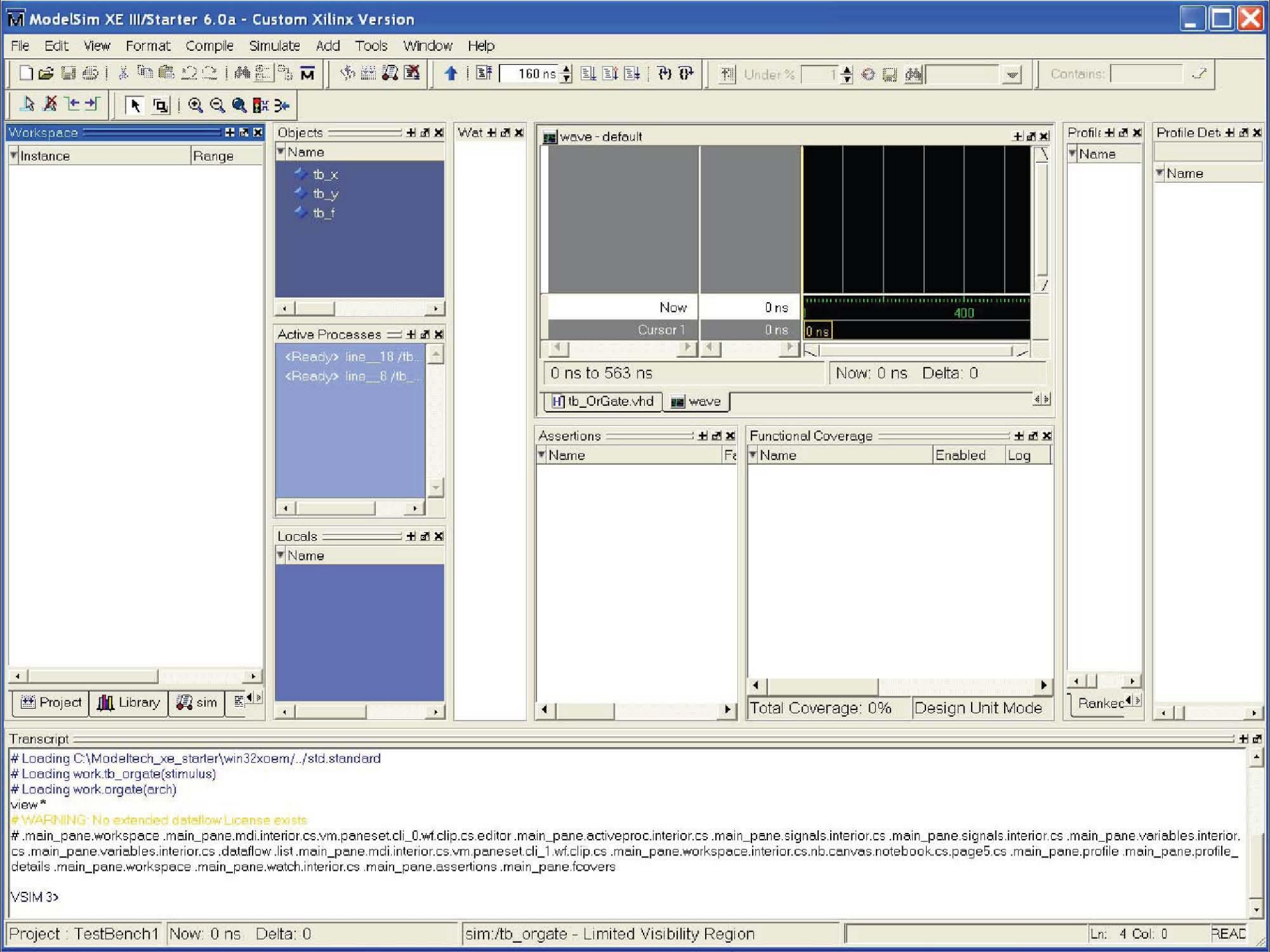


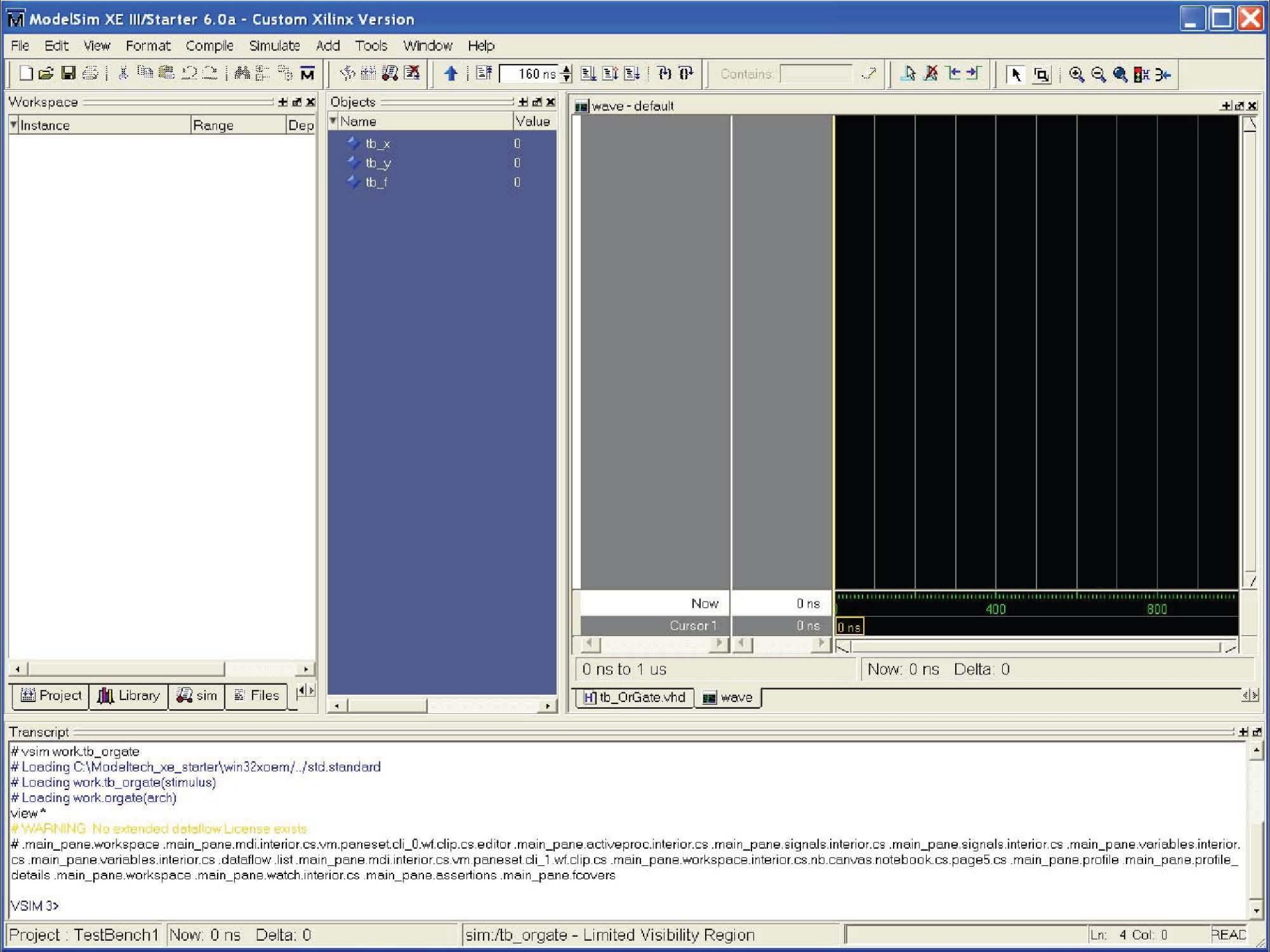


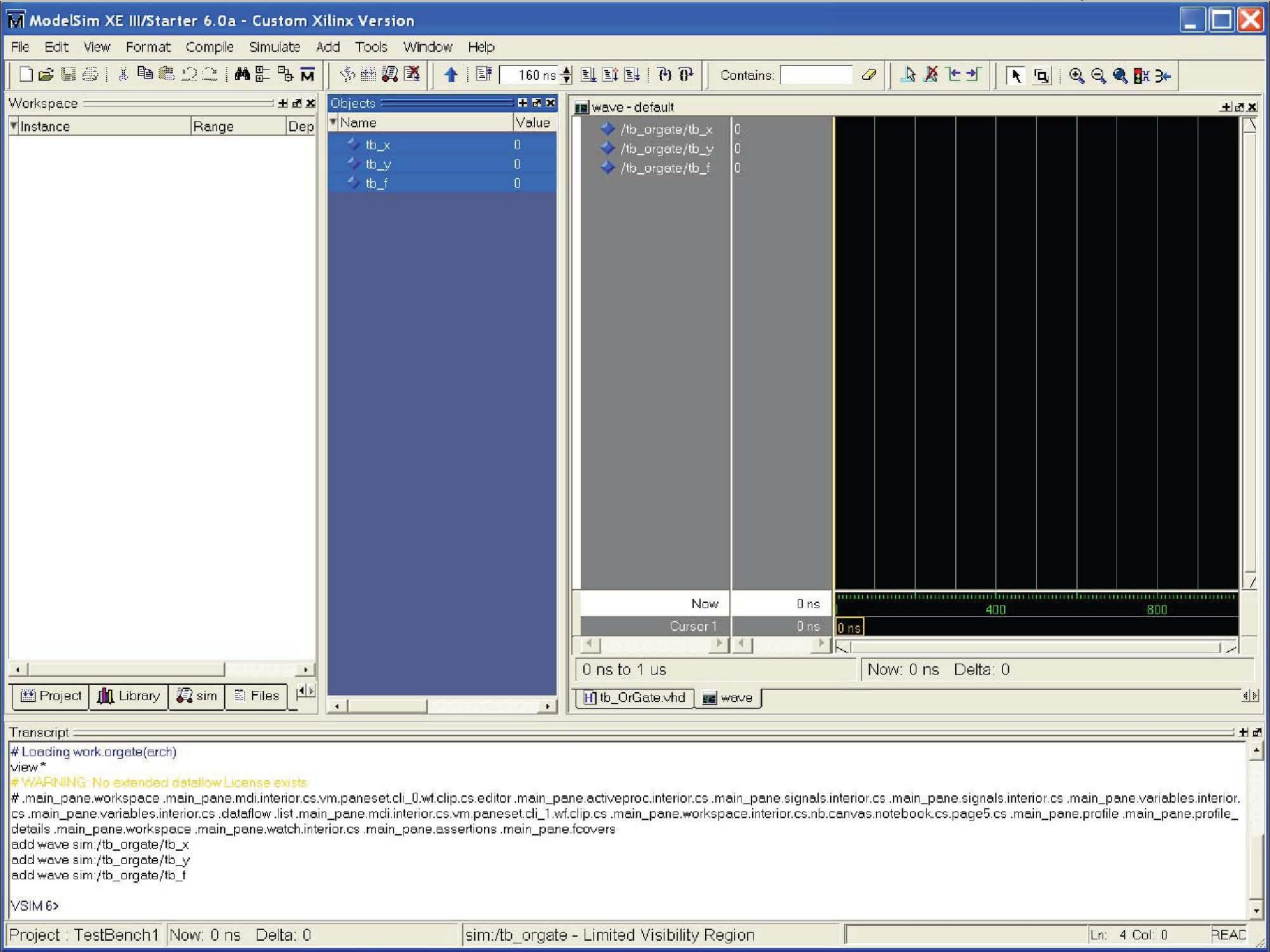












wave - default

```

    /tb_orgate/tb_x 0
    /tb_orgate/tb_y 0
    /tb_orgate/tb_f 0

```



- ☒ List Format
- ☒ Wave Format
- ☒ Breakpoints
- ☒ Logged Signals
- ☒ Virtual Definitions
- ☒ Assertions and Functional Coverage

Cancel

0 ns

□ ns

400

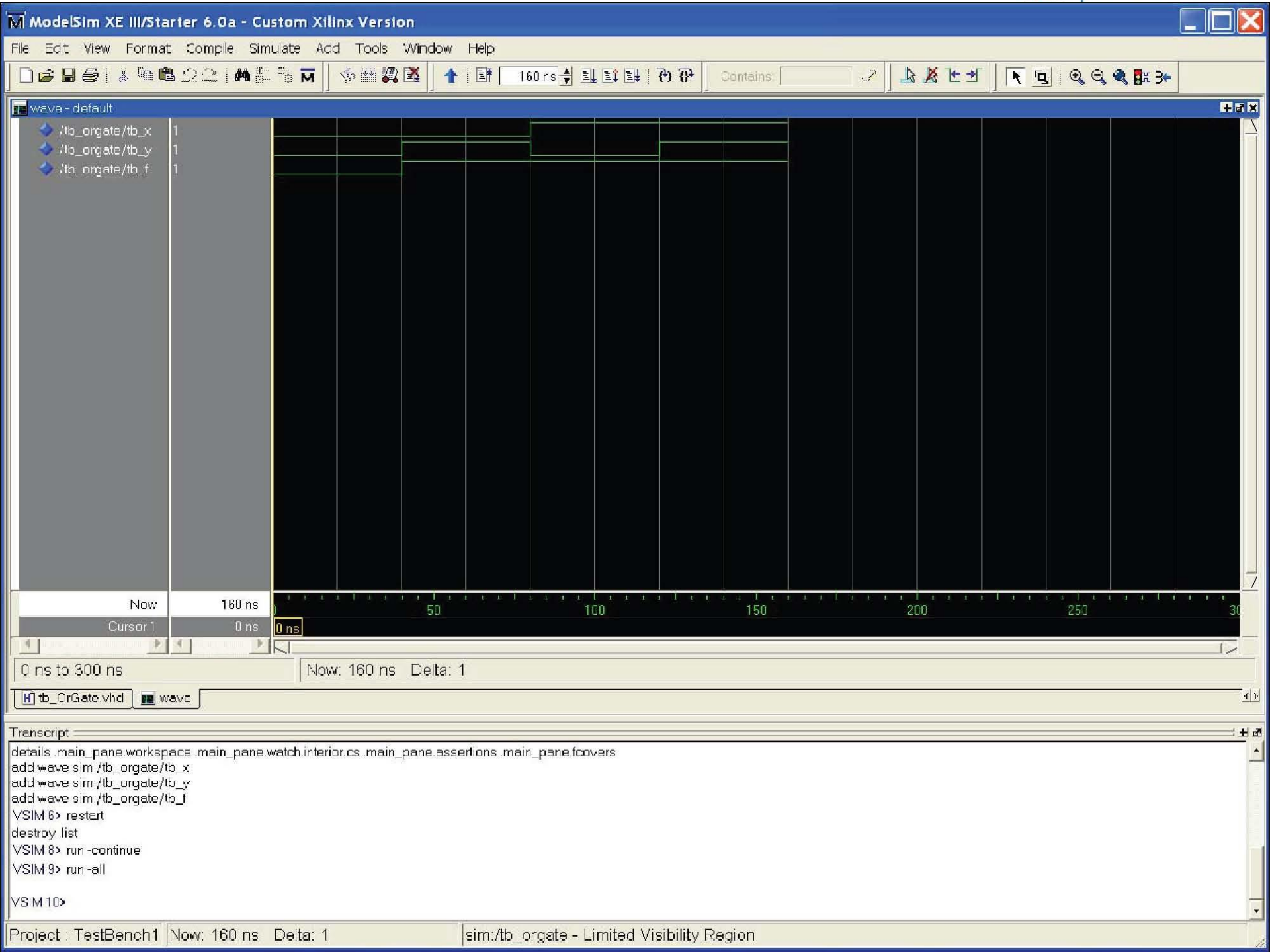
800

1200

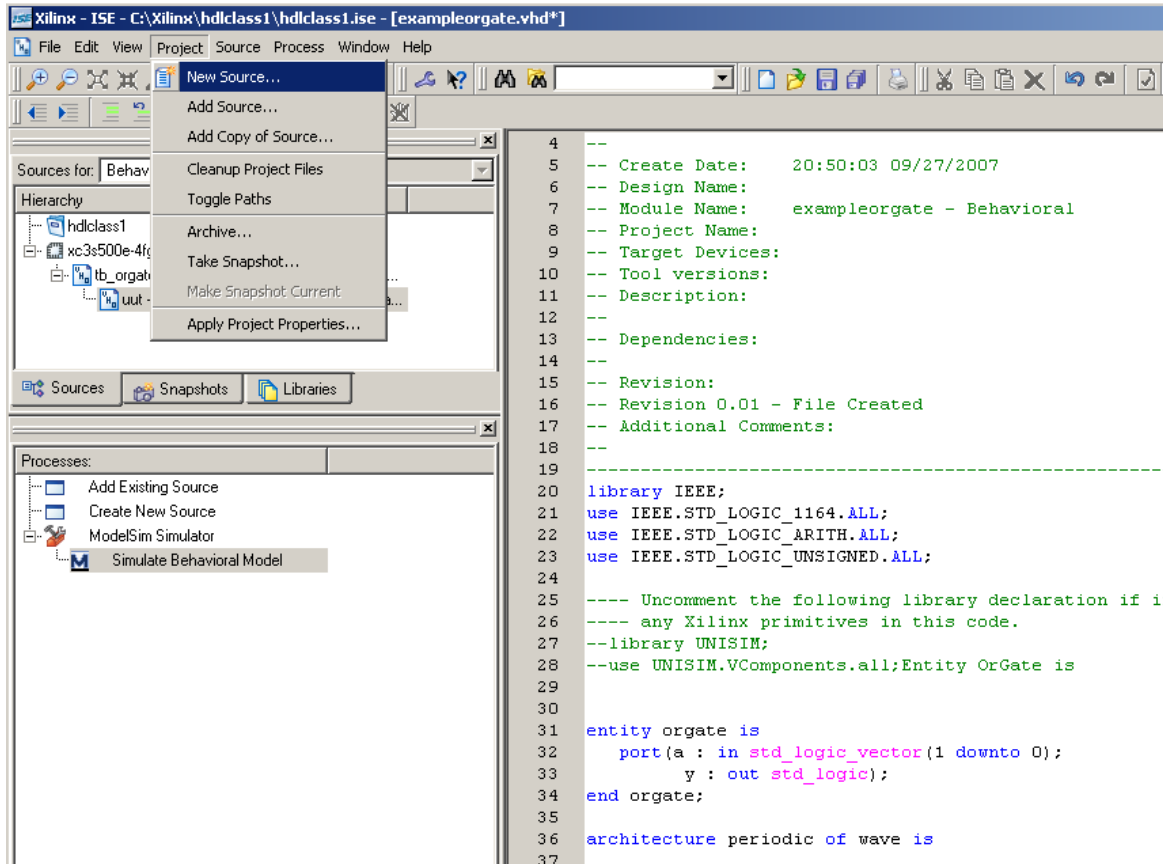
Now: 0 ns Delta: 0

 wave

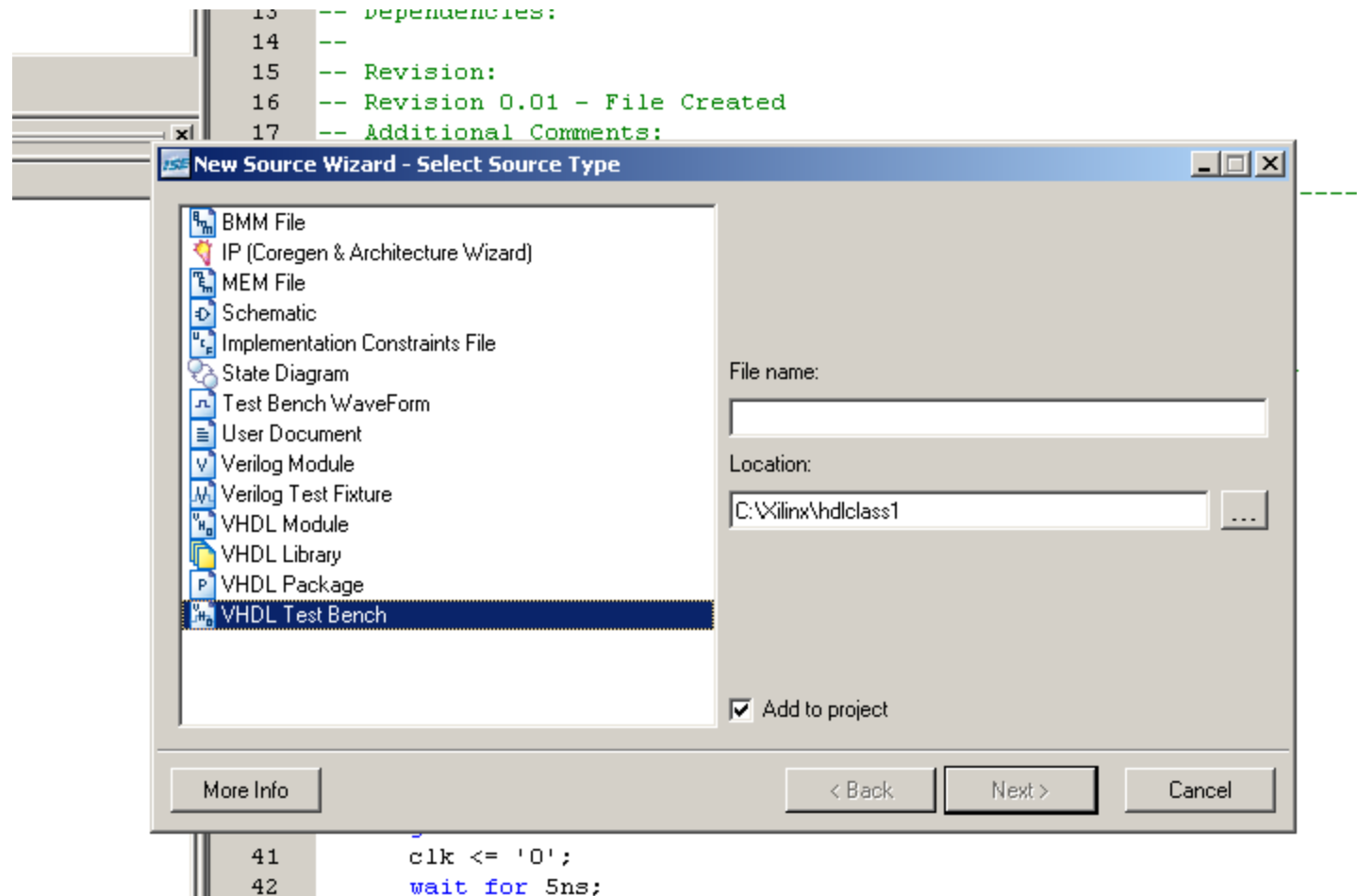
```
view*
# WARNING: No extended dataflow License exists
#.main_pane.workspace .main_pane.mdi.interior.cs.vm.panaset.dl_0.wf.clip.cs.editor .main_pane.activeproc.interior.cs .main_pane.signals.interior.cs .main_pane.signals.interior.cs .main_pane.variables.interior.cs .main_pane.variables.interior.cs .dataflow.list .main_pane.mdi.interior.cs.vm.panaset.dl_1.wf.clip.cs .main_pane.workspace.interior.cs.nb.canvas.notebook.cs.page5.cs .main_pane.profile .main_pane.profile_details .main_pane.workspace .main_pane.watch.interior.cs .main_pane.assertions .main_pane.fcovers
add wave sim:/tb_orgate/tb_x
add wave sim:/tb_orgate/tb_y
add wave sim:/tb_orgate/tb_f
VSIM 6> restart
```



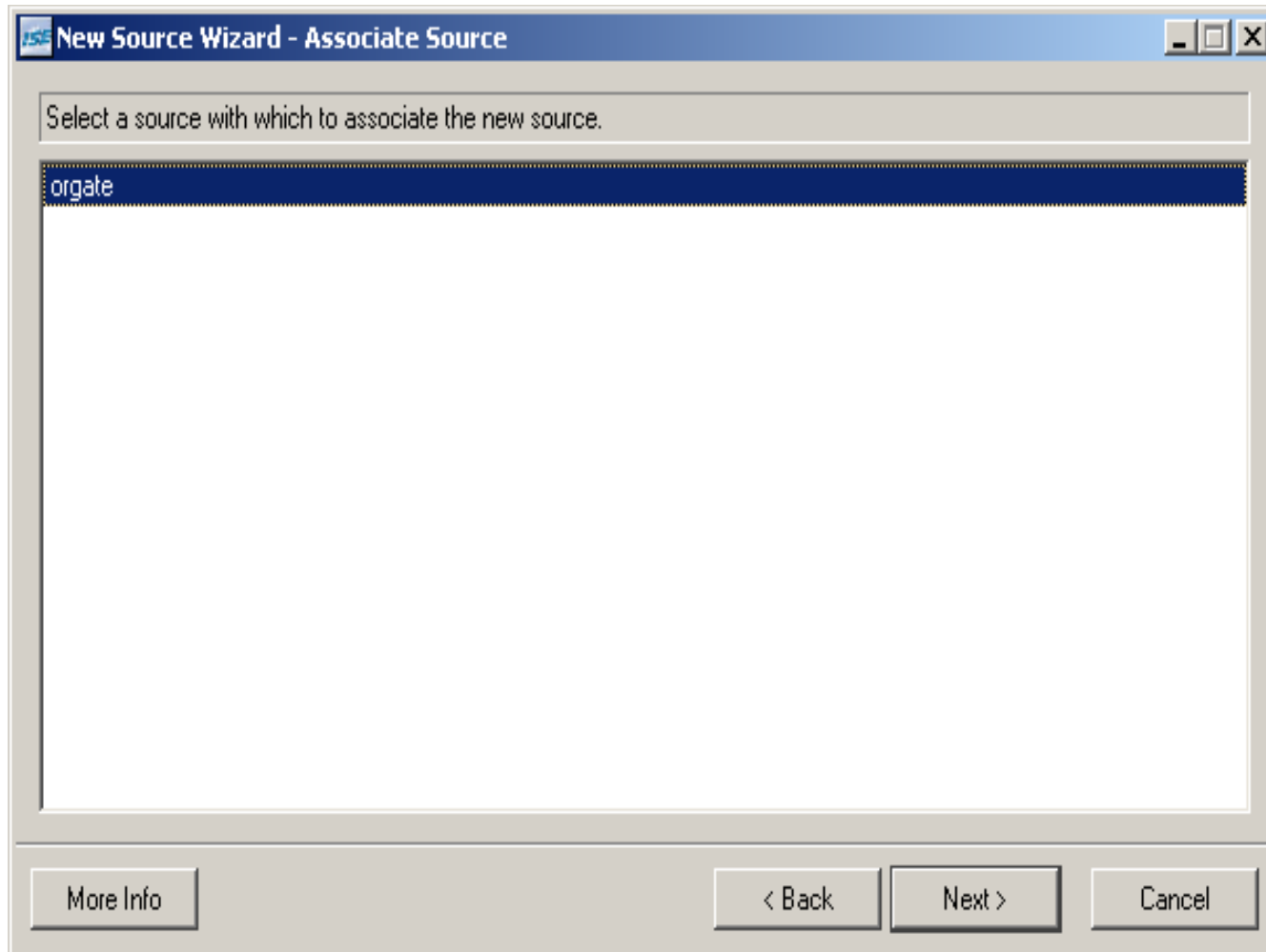
Testbench with Xilinx XST



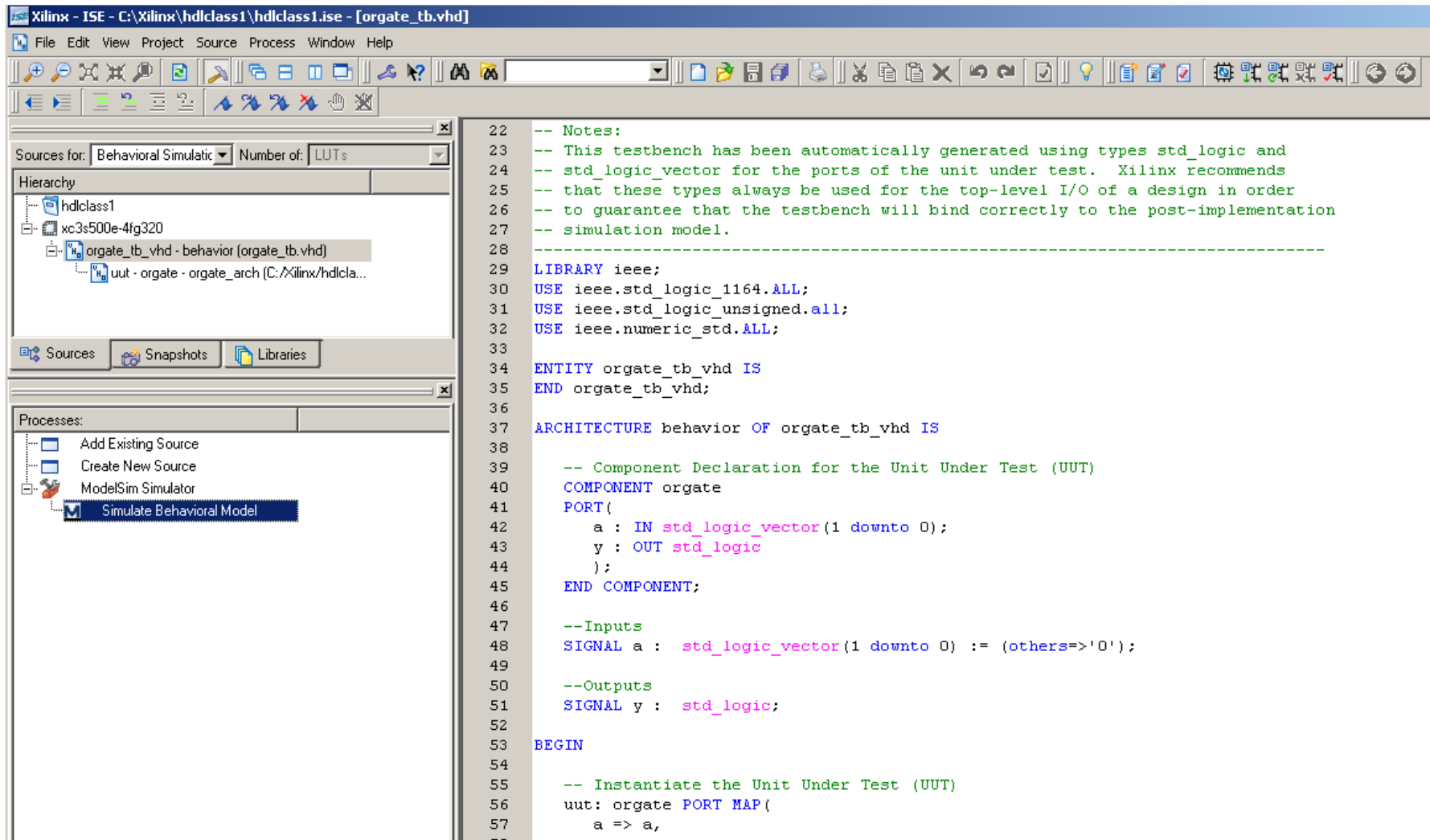
Testbench with Xilinx XST



Testbench with Xilinx XST



Testbench with Xilinx XST



The screenshot shows the Xilinx ISE IDE interface. The top menu bar includes File, Edit, View, Project, Source, Process, Window, and Help. The toolbar contains various icons for file operations, simulation, and project management. The left pane shows the project hierarchy for 'hdlclass1' on a 'xc3s500e-4fg320' device. The 'Sources' tab is active, showing the 'orgate_tb_vhd' file. The 'Processes' pane on the left shows the 'Simulate Behavioral Model' process selected. The main editor window displays the VHDL code for the testbench.

```

22  -- Notes:
23  -- This testbench has been automatically generated using types std_logic and
24  -- std_logic_vector for the ports of the unit under test.  Xilinx recommends
25  -- that these types always be used for the top-level I/O of a design in order
26  -- to guarantee that the testbench will bind correctly to the post-implementation
27  -- simulation model.
28  -----
29  LIBRARY ieee;
30  USE ieee.std_logic_1164.ALL;
31  USE ieee.std_logic_unsigned.all;
32  USE ieee.numeric_std.ALL;
33
34  ENTITY orgate_tb_vhd IS
35  END orgate_tb_vhd;
36
37  ARCHITECTURE behavior OF orgate_tb_vhd IS
38
39      -- Component Declaration for the Unit Under Test (UUT)
40      COMPONENT orgate
41      PORT(
42          a : IN std_logic_vector(1 downto 0);
43          y : OUT std_logic
44      );
45      END COMPONENT;
46
47      --Inputs
48      SIGNAL a :  std_logic_vector(1 downto 0) := (others=>'0');
49
50      --Outputs
51      SIGNAL y :  std_logic;
52
53  BEGIN
54
55      -- Instantiate the Unit Under Test (UUT)
56      uut: orgate PORT MAP(
57          a => a,
58          ..
59

```

TestBench With File I/O

- Testbench with File I/O is very useful where a large population of test data is required
- We must add the '**textio**' package at the header, which is included in the standard library

Library std;

use std.textio.**all**;

- **Note:**
 - Although IEEE provides 'textio' package, this package is included in 'std' library rather than 'ieee' library in ModelSim
 - So add '**use** std.textio.**all**;' instead of '**use** ieee.textio.**all**;' in the beginning of your VHDL codes for ModelSim simulation

TestBench With File I/O

- This textio package allows:
 - Open one or more files
 - Read lines from the file
 - Parse lines to form data elements such as arrays or records
- This is combined with standard built in functions for:
 - Opening
 - Reading from file
 - Writing to file
- VHDL1076-1987 File I/O
 - open and read are simultaneous
- VHDL1076-1993 File I/O
 - Open and read operations are separate
 - Better control over the opening and closing of files

TestBench With File I/O

File I/O syntax

VHDL'87

--Declare a file identifier, DataFile, and open a file with
--the file name of testdata.txt

file DataFile: text **is in** “testdata.txt”;

VHDL'93

--Declare a file identifier, DataFile

file DataFile: text;

--Open testdata.txt

file_open (DataFile, “testdata.txt”, READ_MODE);

--Close file

file_close (DataFile);

TestBench With File I/O

- We shall only describe VHDL'93 File I/O format for our testbench
- The same OR gate design will be used here
- This time, we are going to feed the OR gate inputs, X and Y with binary data from an input file, and the outputs will be saved into an output file
- See the testbench VHDL code from next slide

TestBench Example - 2

```
Library std;
Use std.textio.all;
Entity tb_OrGate is                --no external signals since generated internally
end entity tb_OrGate;
Architecture stimulus of tb_OrGate is
--declare the component for the design under test
Component OrGate
Port (X,Y: in bit;
      F: out bit);
end component;
--declare stimuli and output signals
Signal tb_X,tb_Y, tb_F:bit;
Constant period: time:=40ns;
File stim_in: text;                --declare a text file with stim_in
File stim_out: text;               --declare a text file with stim_out
```



TestBench Example – 2 (cont...)

begin

--create an instance of the or gate, locals=>actuals

dut: OrGate **port map** (X=>tb_X, Y=>tb_Y, F=>tb_F);

process

Variable infile_line: line; --line is defined in textio

Variable data_in: bit;

Variable outfile_line: line; --line is defined in textio

Variable char_space: character := ' ';

begin

--open a file with READ_MODE

file_open (stim_in,"C:\ExampleVHDLcode\DataIn.txt", READ_MODE);

while not endfile (stim_in) **loop**

Readline (stim_in,infile_line);

--read a line into infile_line from stim_in

Read (infile_line, data_in);

--read the first bit into data_in from infile_line

tb_X <= data_in;

--send this first bit to tb_X

read(infile_line, data_in);

--read the second bit into from infile_line

tb_Y <= data_in;

--send this second bit to tb_y

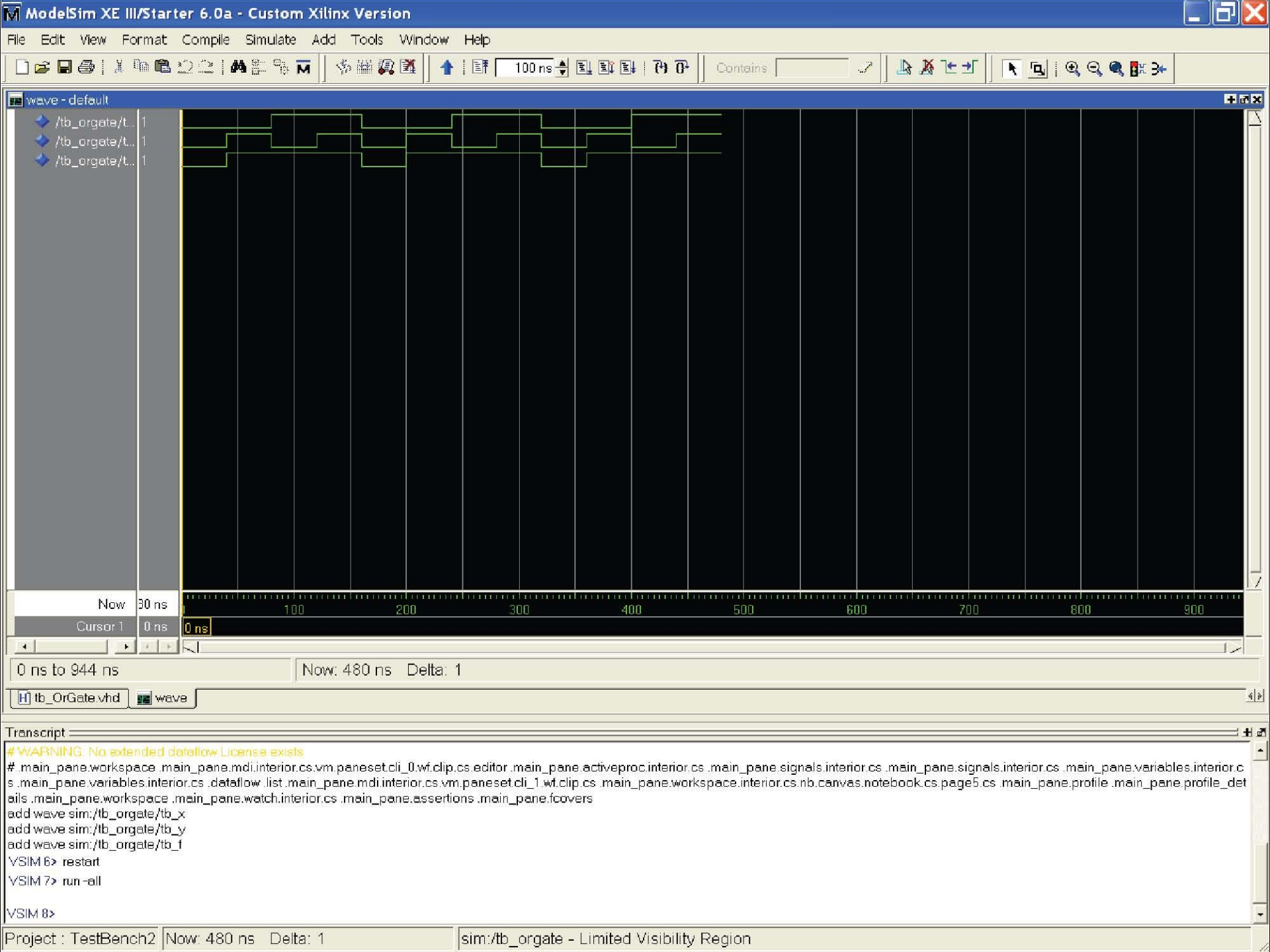
TestBench Example - 2

```
wait for period;
--open a file with APPEND_MODE
file_open (stim_out,"C:\ExampleVHDLcode\DataOut.txt", APPEND_MODE);
write (outfile_line,now);           --write current time instant into outfile_line
write (outfile_line, char_space);   --write a space into outfile_line
write(outfile_line, tb_X);          --write tb_X into outfile_line
write (outfile_line, char_space);   --write a space into outfile_line
write(outfile_line, tb_Y);          --write tb_Y into outfile_line
write (outfile_line, char_space);   --write a space into outfile_line
write(outfile_line, tb_F);          --write tb_F into outfile_line
writeln(stim_out,outfile_line);     --write outfile_line into stim_out
file_close(stim_out);              --close stim_out
end loop;
file_close(stim_in);               --close stim_in
wait;
end process;
end architecture stimulus;
```

TestBench Example - 2

The input file, **DataIn.txt**, is as follows:

```
0 0
0 1
1 0
1 1
0 0
0 1
1 0
1 1
0 0
0 1
1 0
1 1
```



TestBench Example - 2

We have got the output file, **DataOut.txt**, as follows:

40 ns 0 0 0

80 ns 0 1 1

120 ns 1 0 1

160 ns 1 1 1

200 ns 0 0 0

240 ns 0 1 1

280 ns 1 0 1

320 ns 1 1 1

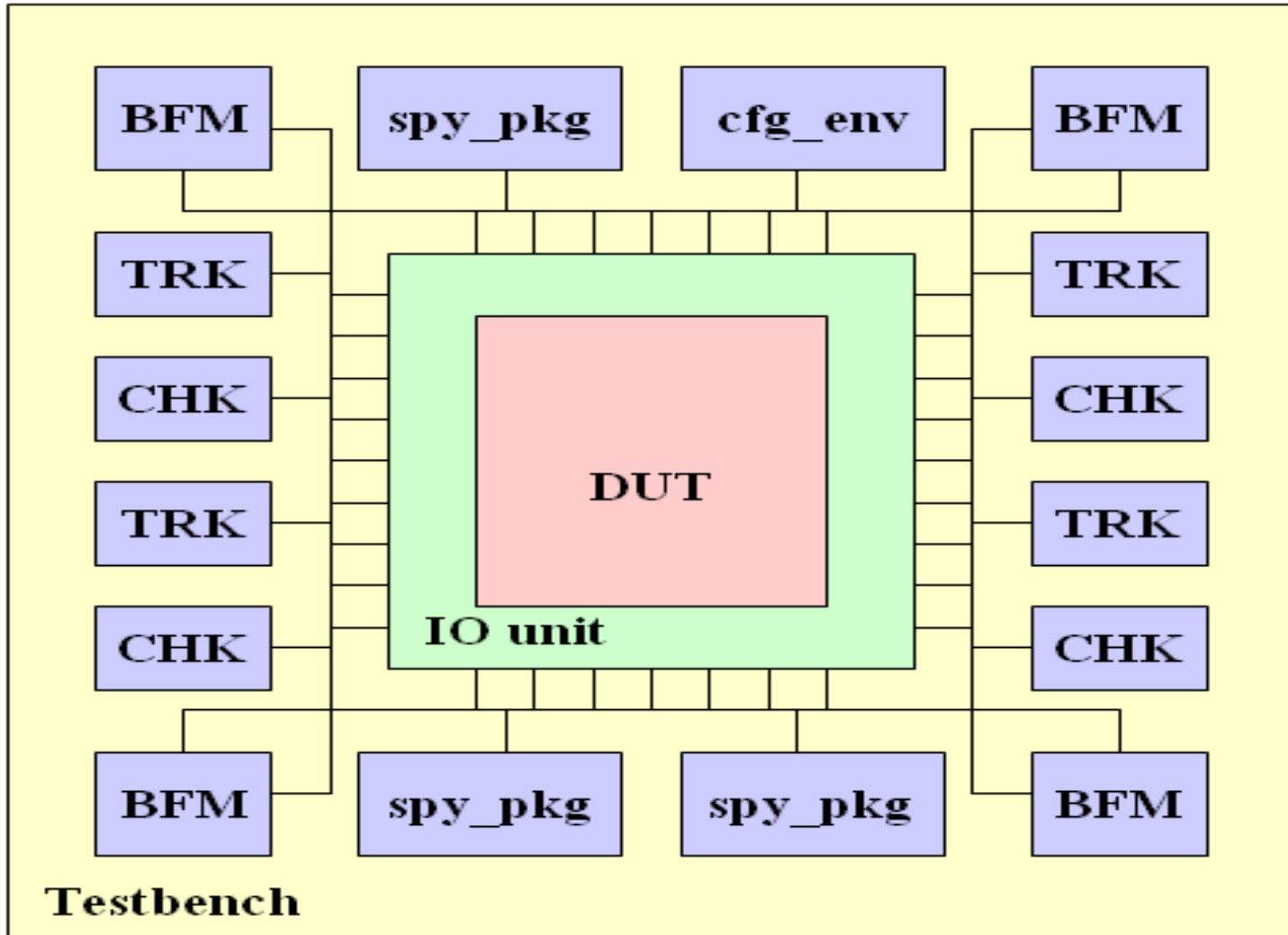
360 ns 0 0 0

400 ns 0 1 1

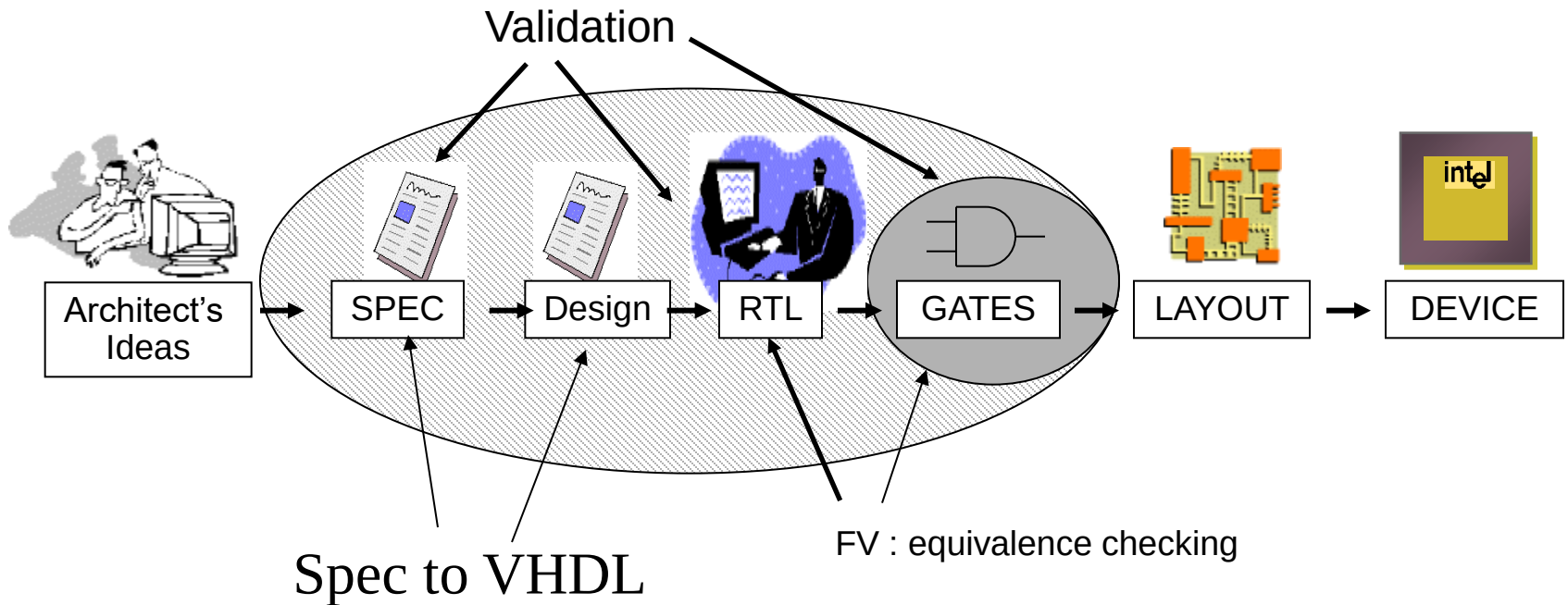
440 ns 1 0 1

480 ns 1 1 1

Model building



Validation Flow



Validation Methods

- Simulation Based
 - A most common method
 - Input vectors applied to the test bench and output vectors are verified with reference values
 - Input and reference vectors can be generated prior to the simulation or on the fly
 - Input vectors (tests) used to test particular functionality of a feature are called directed or focused tests

- Simulation Based (cont...)
 - **Directed test**
 - will not completely focus full design area.
 - **Random test**
 - generated using software
 - an alternative to cover the complete design.
 - consume a lot of validation time in debugging the failed tests.
 - **Coverage tool**
 - used to identify the parts of the design that have not been tested
 - Code coverage - % of the design code being exercised by the test
 - Functional coverage - % of the design functionality being tested
 - Simulators are
 - event based, cycle based and hardware based

- Simulation Based (cont...)
 - **Event Based**
 - A block of statement is validated whenever there is a change in the input of the block.
 - Simulator runs faster at Low event count design.
 - **Cycle Based**
 - Validates the design based on their clock domain
 - With in the given clock domain, the circuit is validated using the triggering edge of the clock.
 - Simulator runs faster for a design having less clock domain.
 - Clock domains must be well defined
 - Unable to validate Asynchronous circuits
 - **Hardware Based**
 - Also called as Emulators
 - Uses FPGAs to mimic the gate design
 - It can be either event driven or cycle based
 - Slow



- Formal Method Based
 - It doesn't require test vectors
 - Two types
 - Equivalence checking [FV]
 - Property verification [FPV]
 - Formal Verification
 - To check two implementation of a design are functionally equivalent
 - To check the input spaces of both implementation in a systematic way
 - Used to compare RTL Vs Gate , RTL Vs Layout

- Formal Method Based (cont...)
 - Formal Property Verification
 - Property = a partial specification of the design
 - Apply property over the design
 - Checks the complete design for the points that fail the property
 - It is highly efficient since it checks all the points at a time.
 - A failure to write good property will result in design bug!!!

Hierarchical VHDL design



--FullAdder.VHDL

Entity FullAdder **is** --entity declaration

Port (A,B,CIN :**in** bit;
SUM, COUT:**out** bit);
end FullAdder;

Architecture Structure **of** FullAdder **is** --architecture body
component HalfAdder --local component declarations

port(x,y: **in** bit; S,C:**out** bit);

end component;

signal S1, C1, C2: bit; --internal signal declaration

begin

U1: HalfAdder **port map** (x=>A, y=>B, s=>S1, c=>C1);

U2: HalfAdder **port map** (x=>S1, y=>CIN, s=>SUM, c=>C2);

COUT <= C1 **or** C2;

end architecture;

--HalfAdder.VHDL

Entity HalfAdder **is** --entity declaration

Port (X,Y: **in** bit;
 S, C: **out** bit);

end entity HalfAdder;

Architecture Structure **of** HalfAdder **is** --architecture body
begin

 S <= X **xor** Y;

 C <= X **and** Y;

end architecture;

--tb_FullAdder.VHDL

Entity tb_FullAdder **is**
end entity tb_FullAdder;

--no external signals since generated internally

Architecture stimulus **of** tb_FullAdder **is**

Component FullAdder **is** --component declaration for the design under test

Port (A,B,CIN: **in** bit; SUM,COUT:**out** bit);
end component;

Signal tb_A,tb_B, tb_CIN, tb_SUM, tb_COUT : bit;

Constant period: time:=40ns;

begin

dut: FullAdder **port map**(A=>tb_A,B=>tb_B,CIN=>tb_CIN,SUM=>tb_SUM,COUT=>tb_COUT);

Process

begin

tb_A<='0'; tb_B<='0'; tb_CIN<='0'; **wait for** period;

tb_A<='0'; tb_B<='0'; tb_CIN<='1'; **wait for** period;

tb_A<='0'; tb_B<='1'; tb_CIN<='0'; **wait for** period;

tb_A<='0'; tb_B<='1'; tb_CIN<='1'; **wait for** period;

tb_A<='1'; tb_B<='0'; tb_CIN<='0'; **wait for** period;

tb_A<='1'; tb_B<='0'; tb_CIN<='1'; **wait for** period;

tb_A<='1'; tb_B<='1'; tb_CIN<='0'; **wait for** period;

tb_A<='1'; tb_B<='1'; tb_CIN<='1'; **wait for** period;

wait;

end process;

end architecture;

- Assume that a digital system needs a 50 MHz clock signal for its operation and reference clock is not provided. Write a behavioural VHDL code that can generate a 50MHz clock signal.
- Develop a VHDL code to generate 2.98Hz, provided an input clock of 50MHz.

For the testbench of an n -bit adder shown below:

- **Draw the waveform of the input stimuli with timing.**
- **Rewrite the testbench code by replacing the concurrent code with PROCESS statement.**

```
library IEEE;
use IEEE.std_logic_1164.all;
entity tb_n_bit_adder is
end tb_n_bit_adder;
architecture tb_n_bit_adder_1 of tb_n_bit_adder is
    constant n: natural := 3;
    signal A, B, Sum: std_logic_vector (n-1 downto 0);
    signal Cin, Cout: std_logic;
    component nbit_adder
        Port (A, B: in std_logic_vector (n-1 downto 0) ;
              Cin: in std_logic;
              Sum: out std_logic_vector (n-1 downto 0);
              Cout: out std_logic);
    end component;

begin
dut: nbit_adder port map (A, B, Cin, Sum, Cout);
    Cin <= '0', '1' after 10 ns, '0' after 25 ns;
    A <= "000", "110" after 5 ns, "010" after 17 ns;
    B <= "101", "111" after 15 ns;
end tb_n_bit_adder_1;
```

End of Lecture 3